Technische Hochschule Rosenheim

Neural Speech Processing and Automatic Speech Recognition

Skipping many things - but it's about getting started!

Remember your questions! We'll answer them and place them into context later in the semester.

Agenda

- Speech Recognition
  - History, evaluation (WER) and why ASR got better
  - The task, the data, the fundamental equation
  - Language model recap
  - Decoding: beam search and language models
- Neural Networks for Speech Recognition
  - Connectionist Temporal Classification (CTC)
  - Data Augmentation
  - RNN-T, Conformer
  - wav2vec 2.0 and self-supervised learning
  - Whisper and speech LLMs
- Beyond Transcription
  - Speaker Recognition and Diarization
    - ECAPA-TDNN
  - Paralinguistics

Automatic Speech Recognition

Related Problems

Automatic speech recognition

- spontaneous speech vs read speech
- Large vocabulary vs small vocabulary
- Speaker-dependent vs speaker-independent
- Noise robustness
- Low resource languages
- Accent robustness

Text to speech

- low resource
- realistic voice

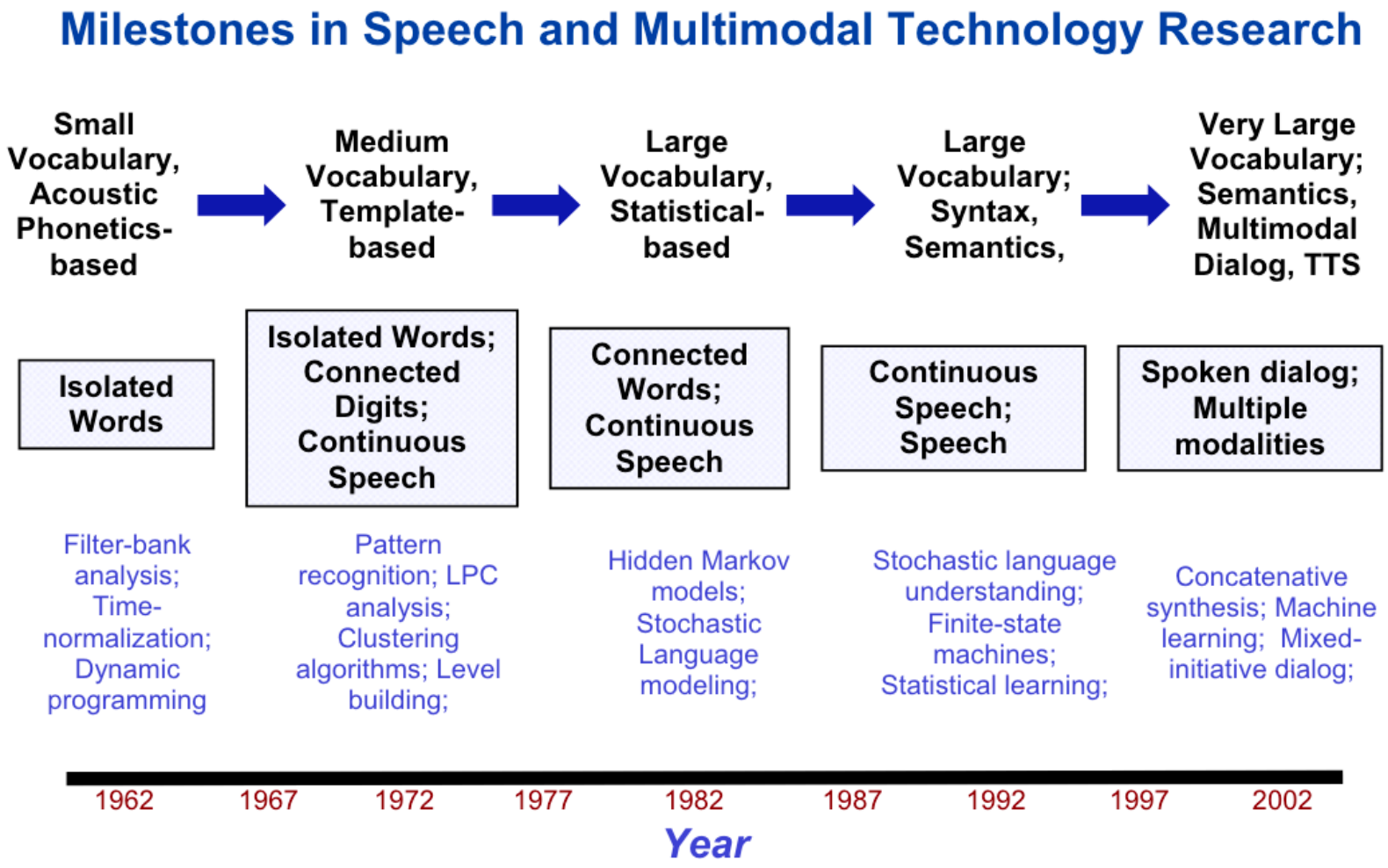
Speaker ID

Speaker diarization

Speech separation

Speech enhancement

History of Speech Recognition



Evaluating ASR Systems

Word Error Rate (WER)

⊙ Word Error Rate (WER)

The **edit distance** (substitutions + deletions + insertions) between the predicted and reference word sequence, divided by the number of reference words. Lower is better; **0 = perfect**. Note it can exceed 100% (lots of insertions). This is literally the Levenshtein distance from Lecture 02 applied to words.

- WER is a common metric used to evaluate the performance of an automatic speech recognition (ASR) system.
- It measures the percentage of words that are incorrectly predicted by the ASR system.
- The WER is calculated using the following formula:

$$\text{WER} = \frac{S + D + I}{N}$$

- Where:
  - S is the number of substitutions.
  - D is the number of deletions.
  - I is the number of insertions.
  - N is the total number of words in the reference.

Phone Error Rate (PER)

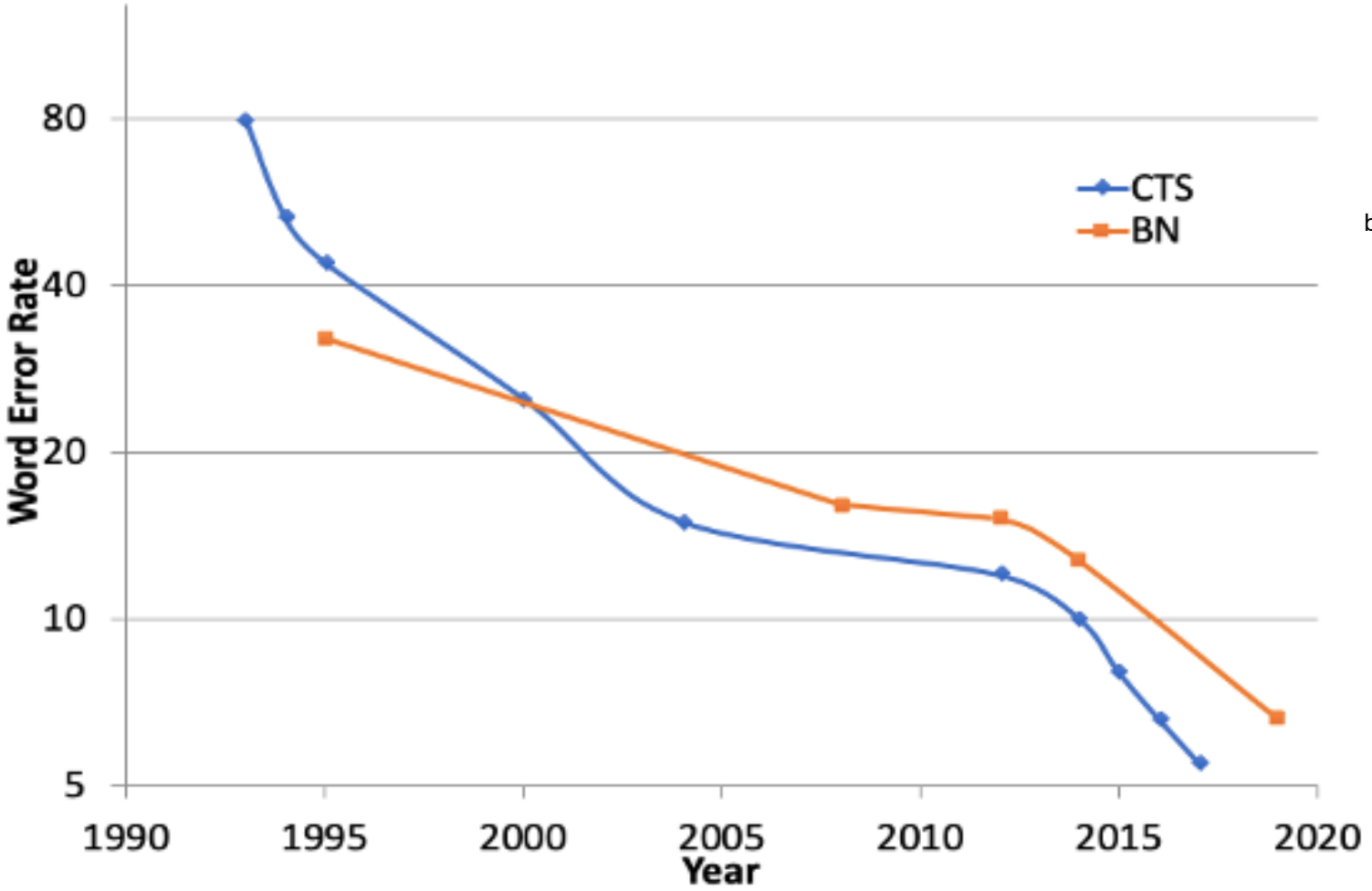
- PER is a similar metric to WER but is used to measure the accuracy of phonetic transcriptions instead of words.
- PER is useful in contexts where phonetic accuracy is more important than word accuracy, such as in phonetic studies or low-resource languages.
- The PER is calculated using the following formula:

$$\text{PER} = \frac{S + D + I}{N}$$

- Where:
  - S is the number of phoneme substitutions.
  - D is the number of phoneme deletions.
  - I is the number of phoneme insertions.
  - N is the total number of phonemes in the reference.

Development of Performance of ASR Systems

Massive Improvement in the last 20 years



captured telephone speech?

broadcast news

Is ASR solved in your opinion and perception?

Partially solved, but not there, yet!

- high recognition rates for single speaker but still not solved for:
- Conversation
- Multiple speakers (Cocktail party)
- Spontaneous voice
- Backchannelling noises of acknowledgment
- non-norm speakers

Noise Conditions: Environment, Situation, Room..

Bias: Works worse on underrepresented groups

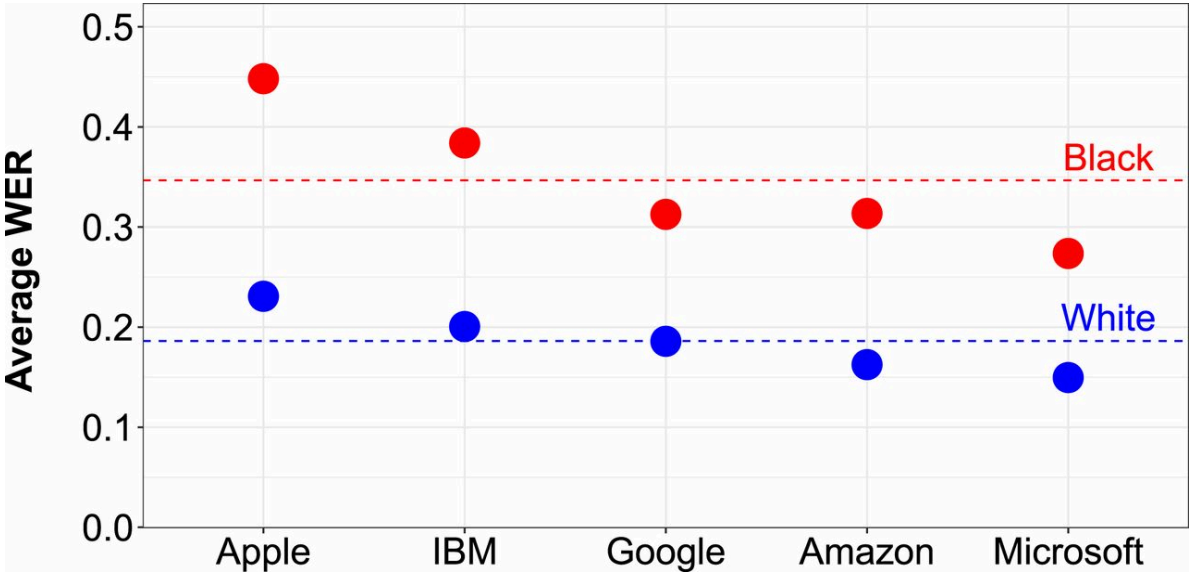
Automatic Speech Recognition

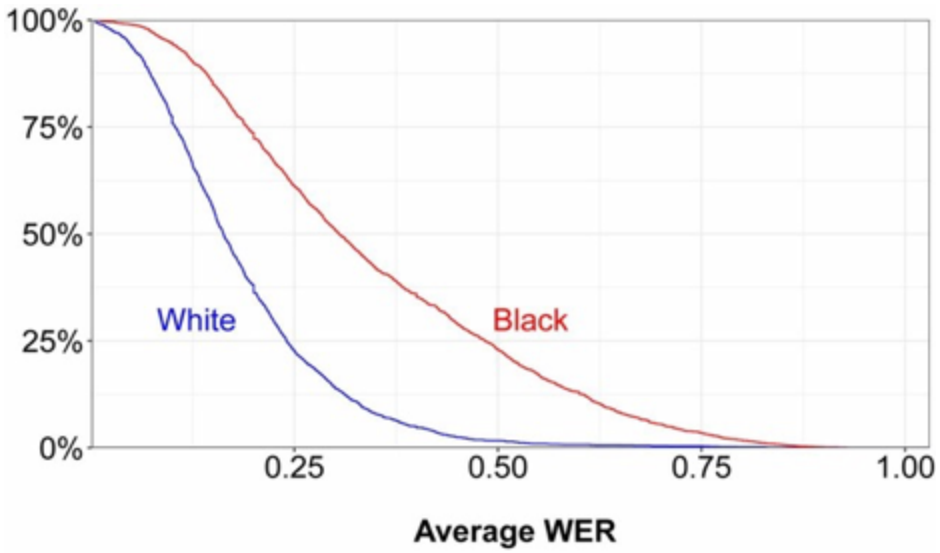
Feng, S., Kudina, O., Halpern, B.M., Scharenborg, O., 2021. Quantifying Bias in Automatic Speech Recognition.

"...state-of-the-art (SotA) ASRs struggle with the large variation in speech due to e.g., gender, age, speech impairment, race, and accents"

Bias in ASR

Koenecke, A., Nam, A., Lake, E., Nudell, J., Quartey, M., Mengesha, Z., Toups, C., Rickford, J.R., Jurafsky, D., Goel, S., 2020. Racial disparities in automated speech recognition.





Why has ASR gotten so much better (since 2012)?

Pre 2012 ASR system:

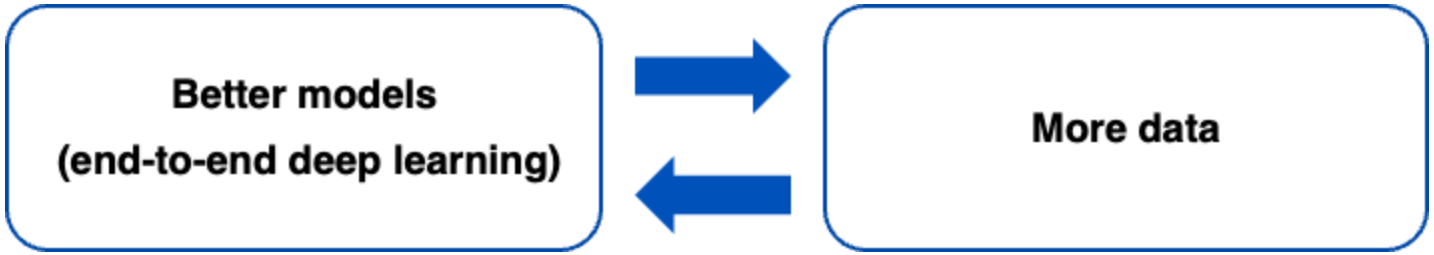
- **Alphabet soup**: Many hand-engineered components
- **Data**: Small and not useful
  - few speakers
  - few communication situations
  - Different data for different components
- **Cascading errors**: Combine modules only at inference
- **Complex**: Difficult to find problems and do research

from speech to recorded speech to feature extraction to acoustic extraction via acoustic model to extracting words from those acoustics to sentence understanding via language model

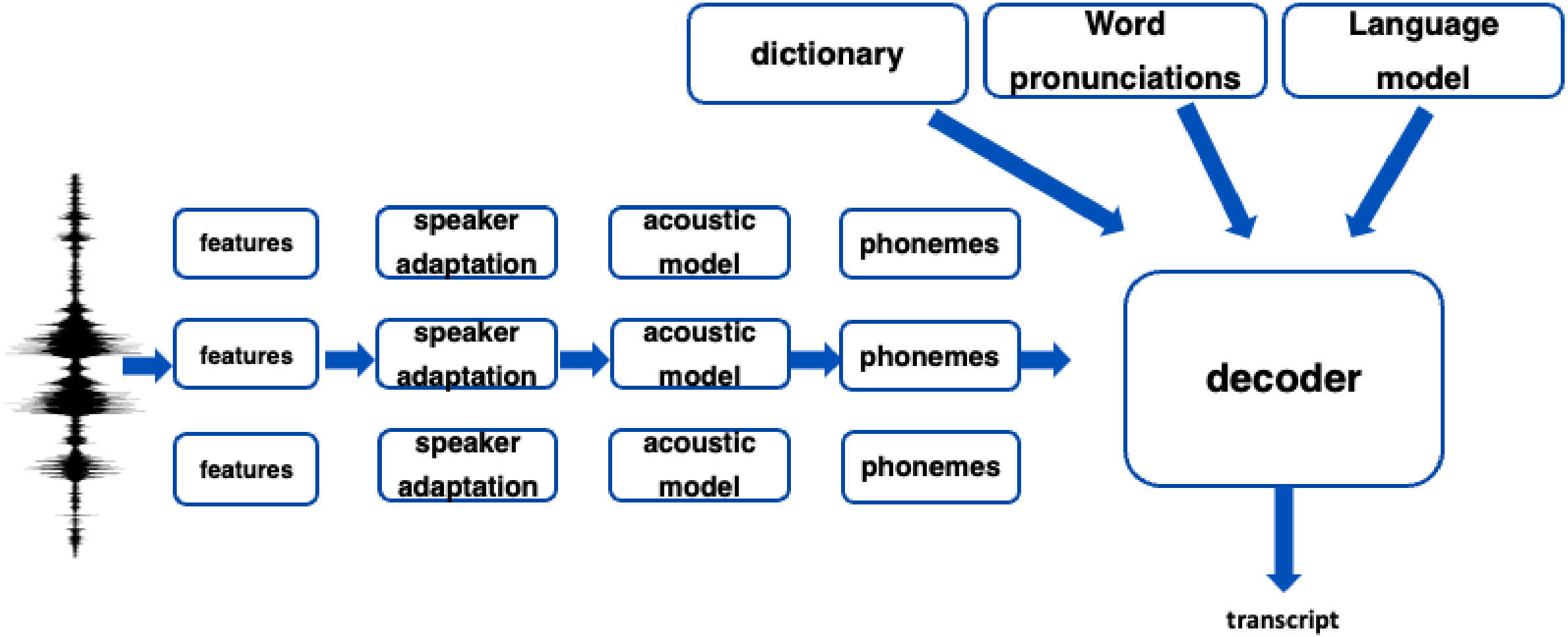
optimization of each individual component but not as a whole?

x

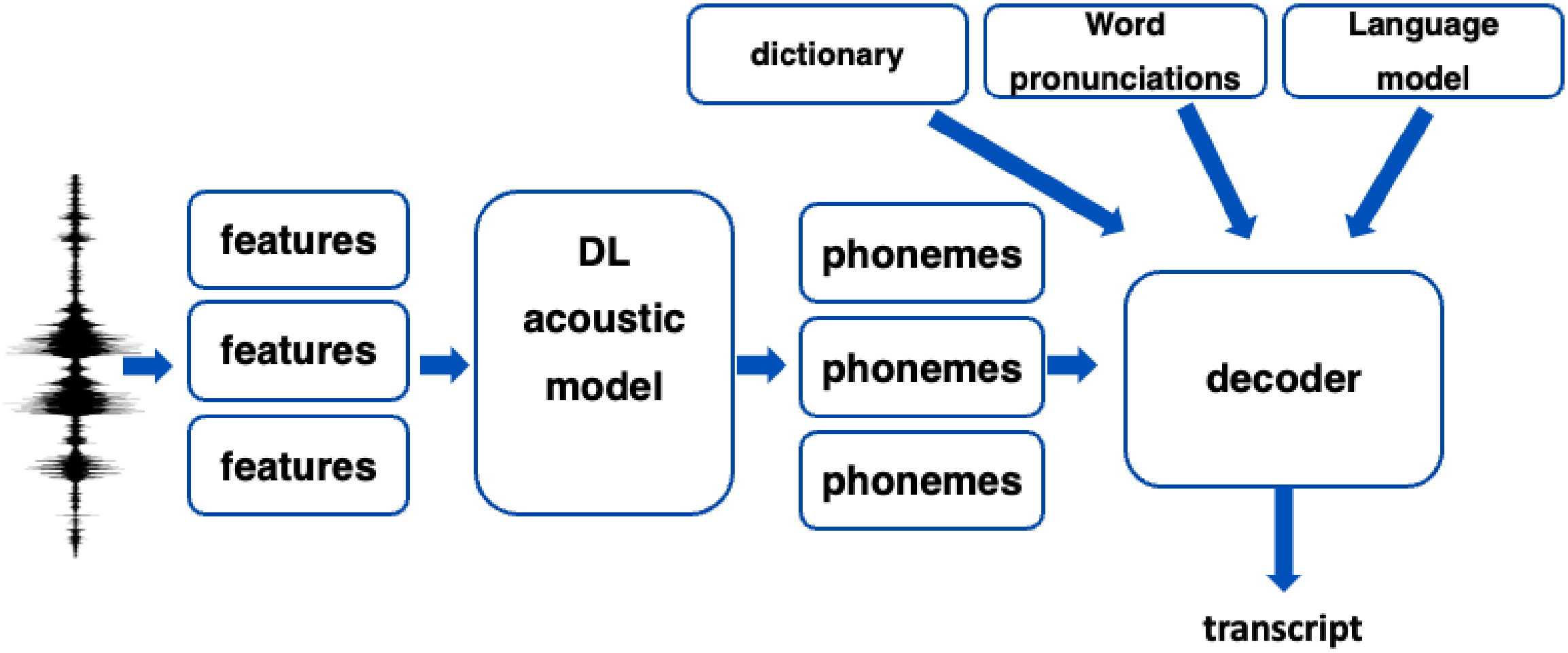
Why has ASR gotten so much better?



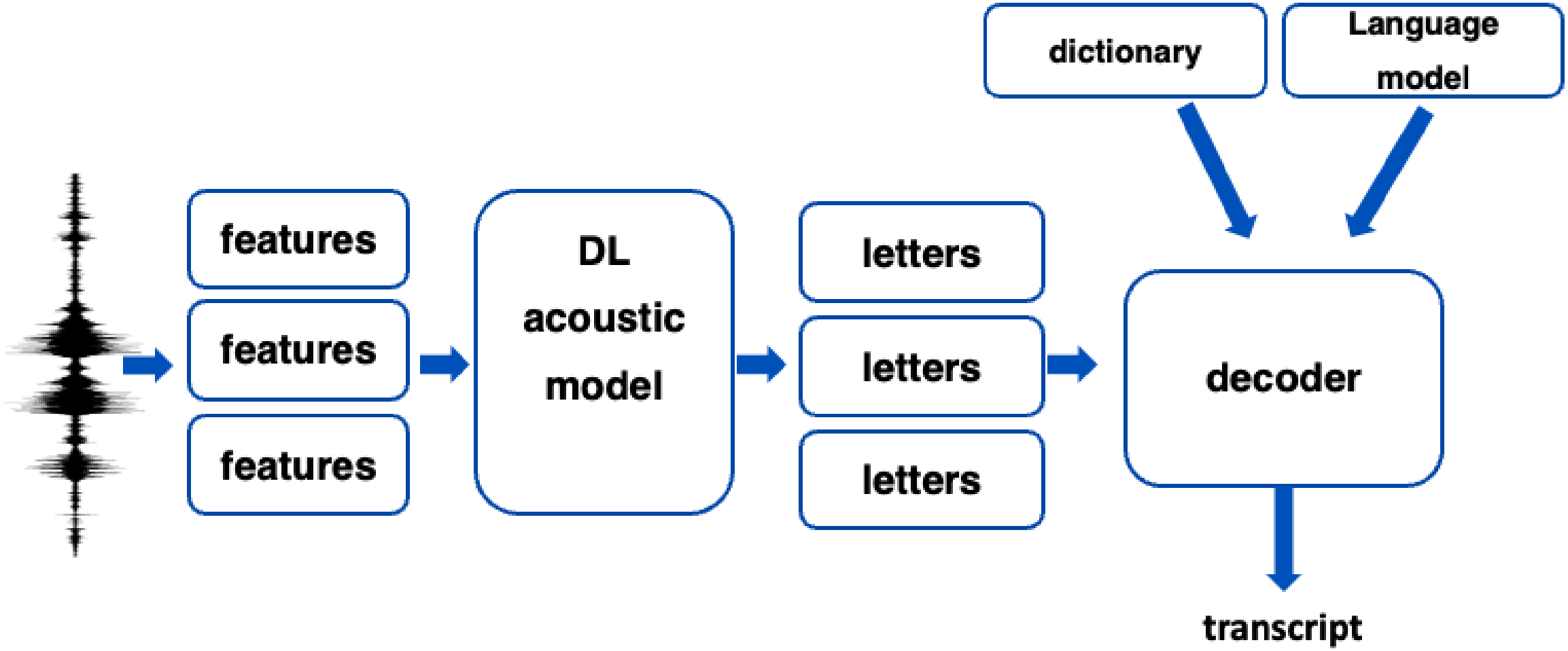
Why has ASR gotten so much better?



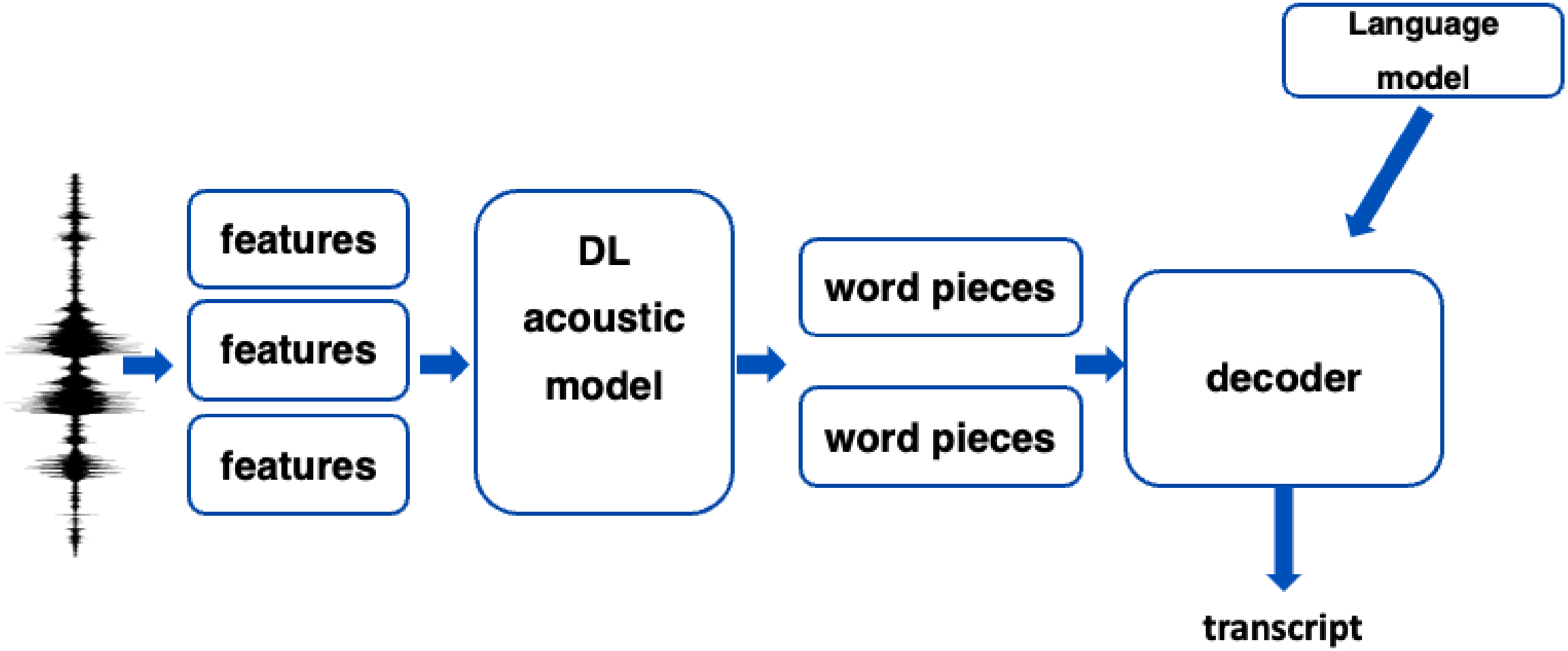
Why has ASR gotten so much better?



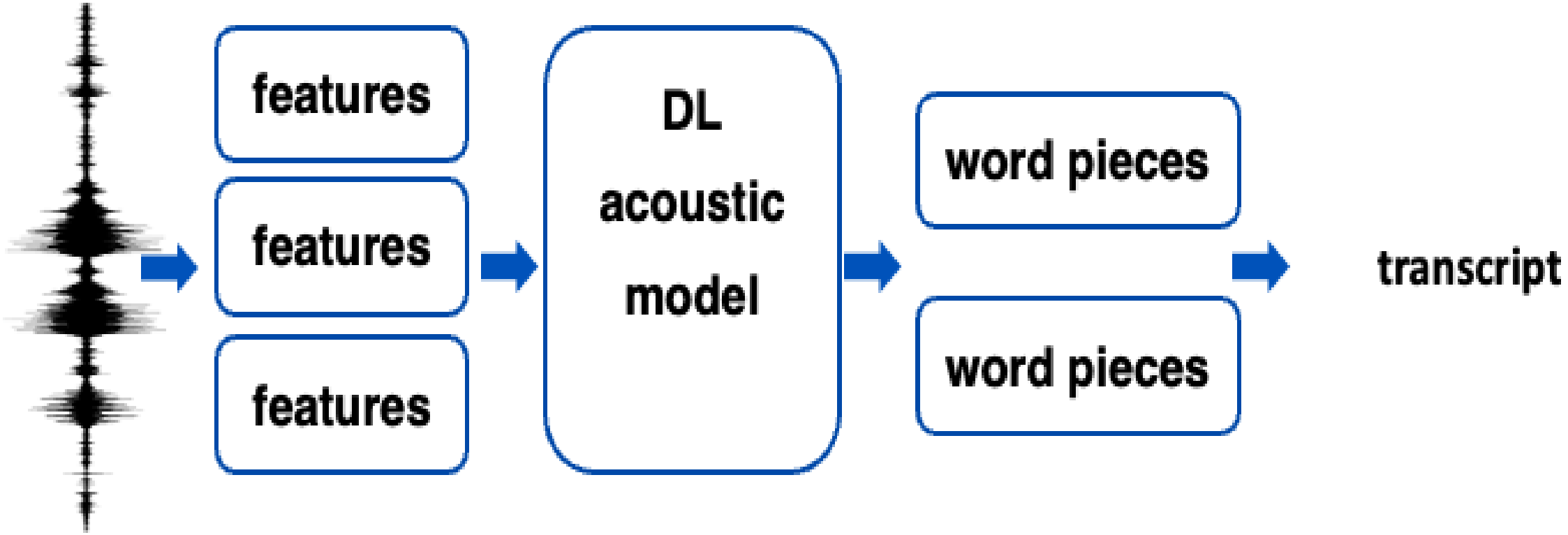
Why has ASR gotten so much better?



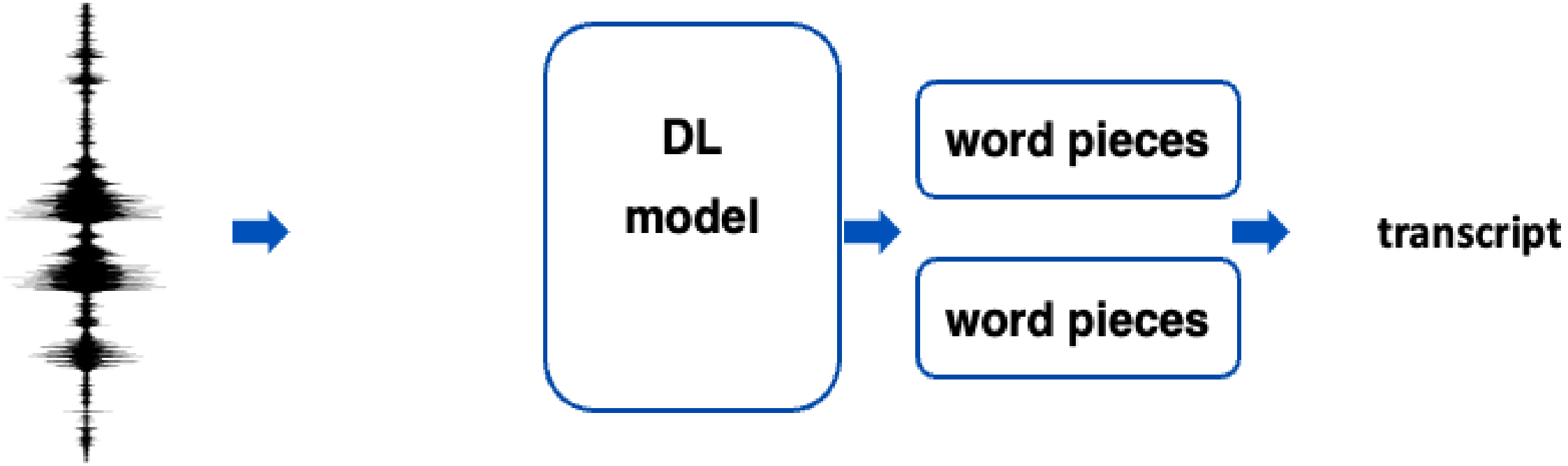
Why has ASR gotten so much better?



Why has ASR gotten so much better?



Why has ASR gotten so much better?



Key Takeaways: Why Modern ASR Works

- ASR is **not yet fully solved**: still struggles with conversation, overlapping speakers, spontaneous/non-norm speech, and noise
- **Bias is real and measurable**: error rates are much higher for underrepresented groups (race, age, accent, impairment) — an ethical, not just technical, problem
- The post-2012 leap came from **3 things together**: (1) **end-to-end deep learning** replacing hand-engineered pipelines, (2) **massively more data**, (3) **GPU compute**
- Old pipelines failed from **cascading errors** and **module separation**; one jointly-trained network avoids both

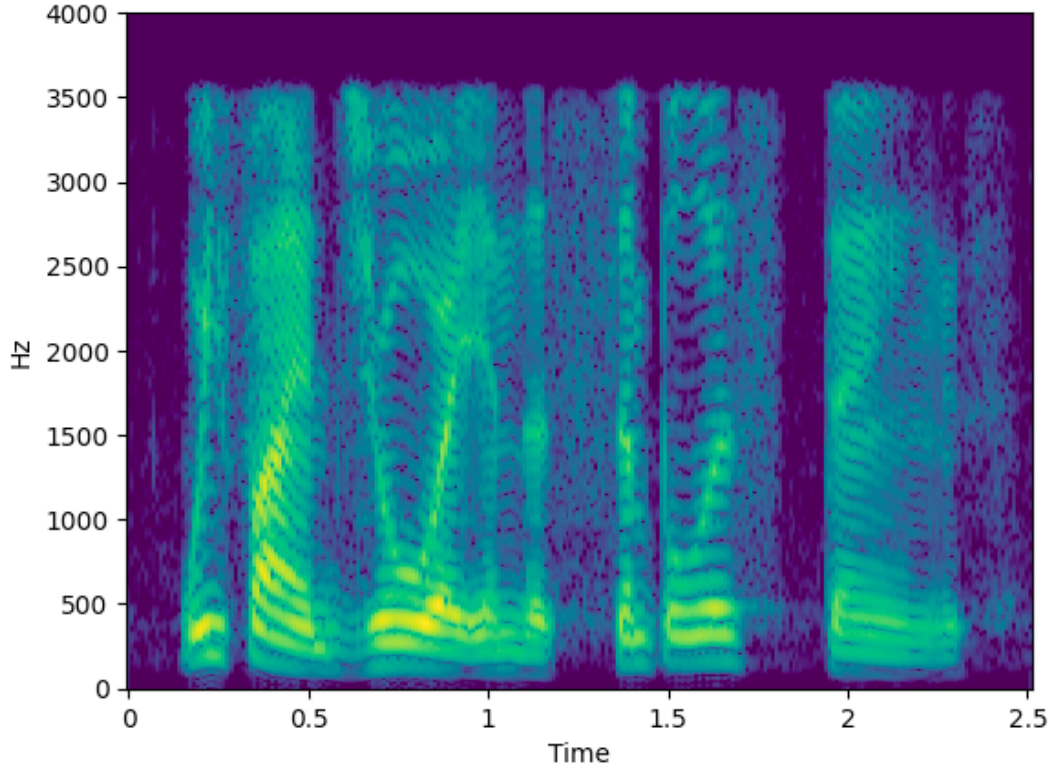
Features of Modern ASR Systems

Speech representation

- Capturing what allows us to understand speech
- Discussed in the last lectures:
  - Human speech is ~85Hz - ~8kHz
  - Telephone speech is sampled at 8kHz: 300Hz - 4kHz bandwidth
- Speech processing mostly around 16kHz 16-bit/ sample PCM

Speech representation for ASR

- Spectrograms -> high dimensional
- Mel frequency spectrograms -> lower dimensional, mapped to a perceptual scale
- log-mel features -> log of the mel spectrogram
- Most common configurations for speech recognition:
  - frame width: 25ms
  - frame step: 10ms



Mel Frequency Cepstral Coefficients (MFCCs)

- discrete cosine transform of the log-mel features
- lower dimensional representation of the log-mel features
- discrete cosine transform of the log-mel features
- delta and delta-delta coefficients

Speech Recognition Training Data

Traditionally:

- Matching transcriptions to audio for sentences (in ASR called utterances)
- phonetic lexicon (pronunciation dictionary)
- Sentences composed of words
- minimal unit is a "phoneme"
  - Set of 40-60 distinct sounds
  - vary per language
  - Universal representations: IPA / X-SAMPA (ASCII)

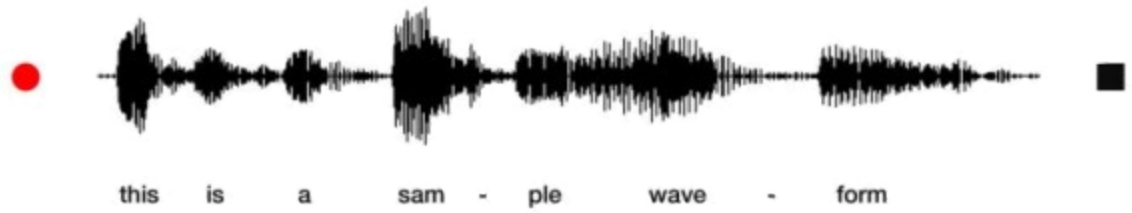
> ⊙ **Phoneme**
>
> The smallest unit of sound that **distinguishes meaning** in a language (e.g. /b/ vs /p/ in *bat* vs *pat*).

Datasets:

- TIMIT ("Toy" dataset)
- LibriSpeech
- Common Voice
- WSJ
- Switchboard

The ASR Problem

- **Continuous Speech**:
  - No clear boundaries between words
  - No clear boundaries between phonemes
  - No clear boundaries between sentences
  - No clear boundaries between speakers



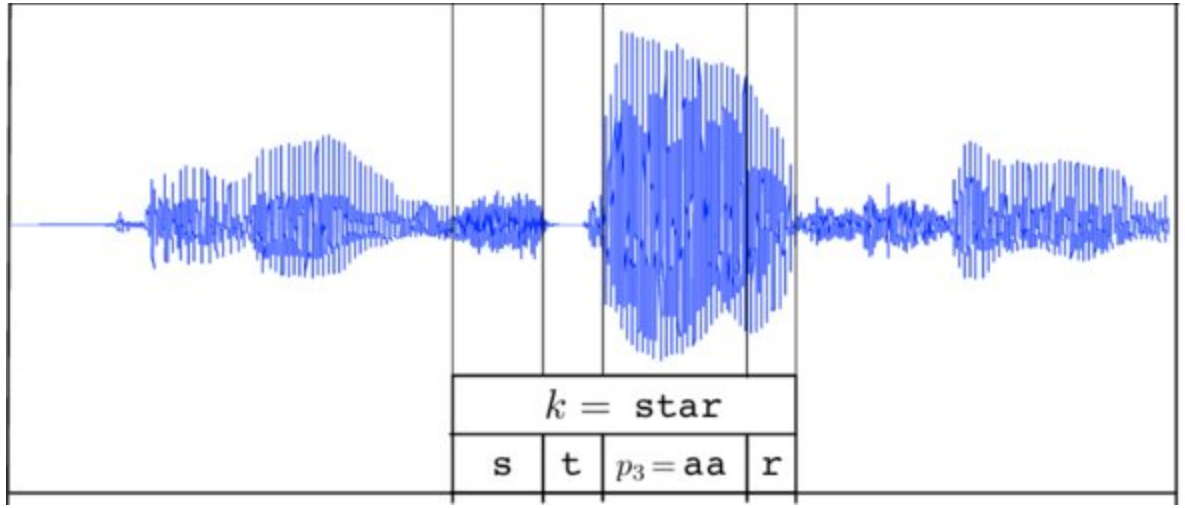
- Large vocabulary (current Systems 300k-2M)
- Domain specific language/ vocabulary
- **Wanted**: Word sequence given an audio signal with time alignment

Preparing Training Data

What we have:

- audio files
- transcripts

What we need:



Fundamental Equation of Speech Recognition

- Given an audio signal X made up of observations o, decode (output) the most likely sequence of words $\hat{w}$, from all possible sequences $\sum *$

$$\begin{aligned}\hat{w} &= \arg\max_{w \in \Sigma^*} P(w|o) \\ &= \arg\max_{w \in \Sigma^*} \frac{P(o|w)P(w)}{P(o)} \\ &= \arg\max_{w \in \Sigma^*} P(o|w)P(w)\end{aligned}$$

- What else do we know, that can help us solve this problem?
- The **Language Model**!
- It follows that the most likely sequence of words is the one that maximizes the probability of the word sequence given the audio signal.
- A product of the acoustic model and the language model scores

$$P(o|w) = \sum_{d,c,p} P(o|c)P(c|p)P(p|w)$$

- with p being the phoneme sequence, c the sequence of context dependent states, and the word w

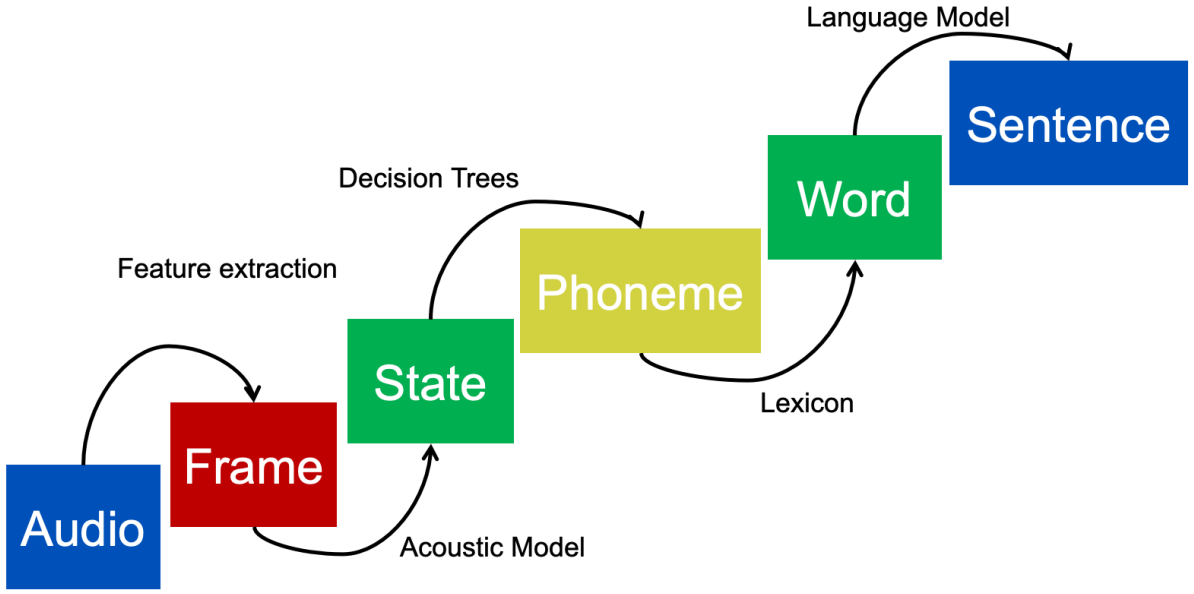
Fundamental equation of Speech Recognition

- Modeling the word sequence with a language model

$$P(w_1, w_2, \ldots, w_n) = P(w_0) \prod (Pw_i | w_0, \ldots)$$

Thinking of ASR as a transduction problem, translating one sequence into another

Traditional view:



Language Models: What are they good for?

- A **language model (LM)** assigns a probability to a sequence of words: $P(w_1, w_2, \ldots, w_n)$
- It prefers fluent, plausible sequences over implausible ones
- Acoustically similar candidates can only be told apart by context:
  - "recognize speech" vs. "wreck a nice beach"
  - "their" vs. "there" vs. "they're"
- The LM can be trained on **text only**, and text is far cheaper than transcribed audio

Language model

A language model scores how likely a word (or token) sequence is. Equivalently, it predicts the next word given the history: $P(w_i \mid w_1, \ldots, w_{i-1})$.

- In ASR it is the $P(w)$ in $\arg\max_w P(o|w)P(w)$

From the Chain Rule to N-grams

- The **chain rule** decomposes the joint probability exactly:

$$P(w_1, \ldots, w_n) = \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1})$$

- The full history is far too long to estimate from counts
- The **Markov assumption** keeps only the last $n-1$ words:

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-n+1}, \ldots, w_{i-1})$$

- **Unigram** ($n = 1$): words are independent, $\prod_i P(w_i)$
- **Bigram** ($n = 2$): $\prod_i P(w_i \mid w_{i-1})$
- **Trigram** ($n = 3$): $\prod_i P(w_i \mid w_{i-2}, w_{i-1})$
- Higher order means more context but also much sparser counts

Estimating N-gram Probabilities

- **Maximum likelihood estimate** from counts, e.g. for a bigram:

$$P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

- Example: if "speech" occurs 100 times and "speech recognition" occurs 40 times, then $P(\text{recognition} \mid \text{speech}) = 0.4$

Problem: zeros and overfitting

Any n-gram that never occurred in the training data gets probability 0, which zeroes out the whole sentence.

Smoothing (add-one, interpolation, backoff, Kneser-Ney) moves some probability mass to unseen n-grams.

- Tools: **SRILM**, **KenLM** (fast, compact, used in many ASR decoders)
- ASR systems typically use word-level 3- to 5-grams trained on very large text corpora

Evaluating a Language Model: Perplexity

- **Intrinsic evaluation**: how well does the LM predict held-out text it has never seen?
- **Perplexity** is the inverse probability of the test set, normalized by its length N:

$$\mathrm{PP}(W) = P(w_1, \ldots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i \mid w_1, \ldots, w_{i-1})}}$$

- **Lower perplexity is better**

Intuition

Perplexity is the **weighted average branching factor**: how many words the model is effectively choosing between at each step.

- A uniform model over a vocabulary of V words has perplexity V
- Always compute it on a held-out test set, never on the training data
- The final judge for ASR is still the **WER** of the whole system, as defined earlier (extrinsic evaluation)

branching factor = how many branches split of

Why does ASR need an *external* Language Model?

- The acoustic model only ever saw **transcribed audio**, which is expensive and limited (hundreds to thousands of hours)
- An LM can be trained on **billions of words of plain text**
- It resolves **homophones and ambiguity** through context
- It can be **swapped per domain or even per user** (medical terms, names, commands) without retraining the acoustic model
- It shapes the **search**: it changes which hypotheses survive during decoding → next section

⊙ Terminology

External LM: a separate model, trained on text, combined with the acoustic model at decoding time.

Internal LM: the language-modelling ability that an end-to-end network (e.g. an RNN-T prediction network or an attention decoder) learns implicitly from its transcripts.

Technische Hochschule Rosenheim

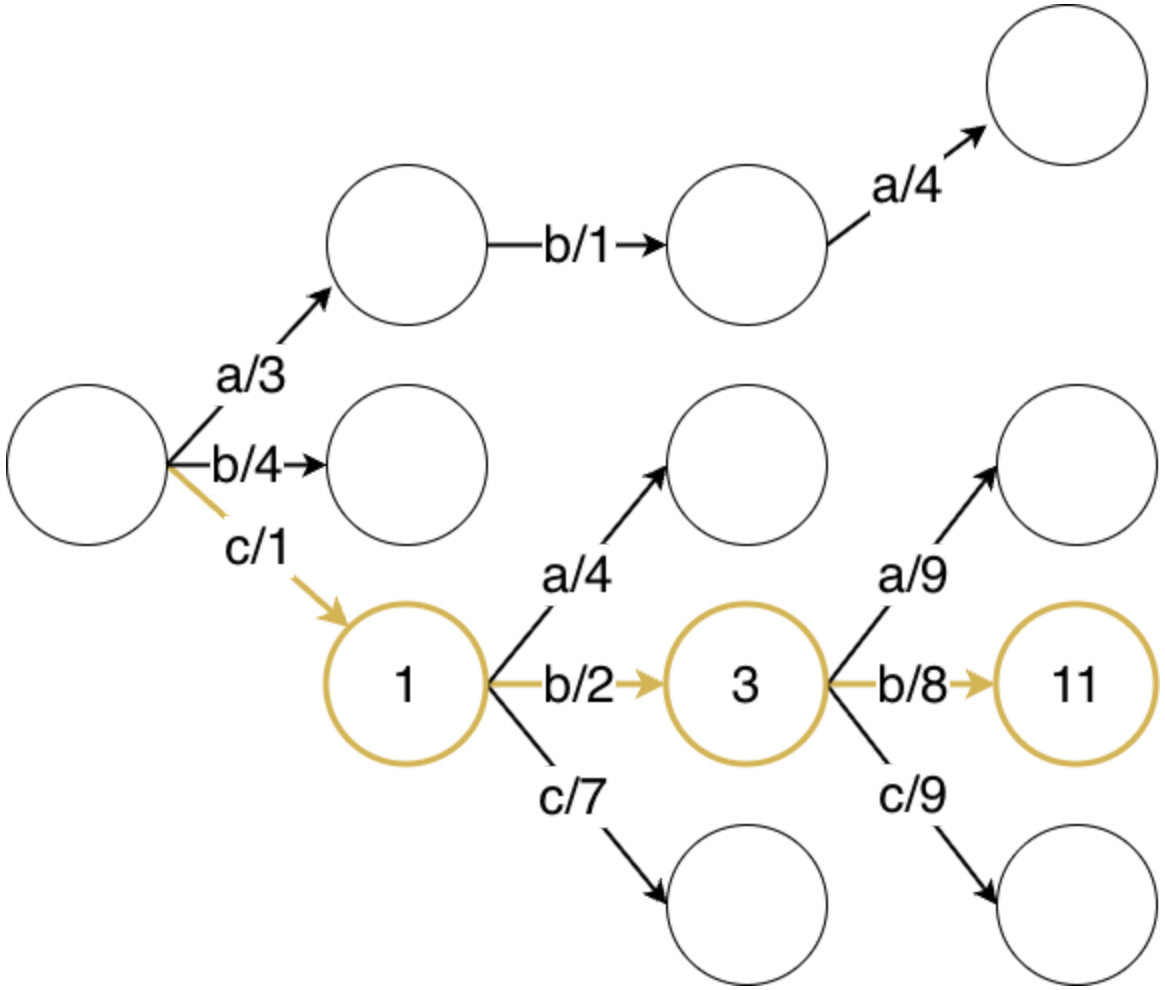
BEAM SEARCH DECODING

Inference

- **Goal:** find the best Y (transcription) given an X (speech). There are two models:
  i. Acoustic model: $\log P(Y \mid X)$
  ii. Language model: $\log P(Y)$
- **Find:** $Y^* = \arg\max_Y \log P(Y \mid X) + \log P(Y)$
- **Language model:** $\log P(Y)$
- Trained on a much larger text corpus
- Fine-tuned for a given application (or even user!)
- Typically a word-level n-gram with n between three and five

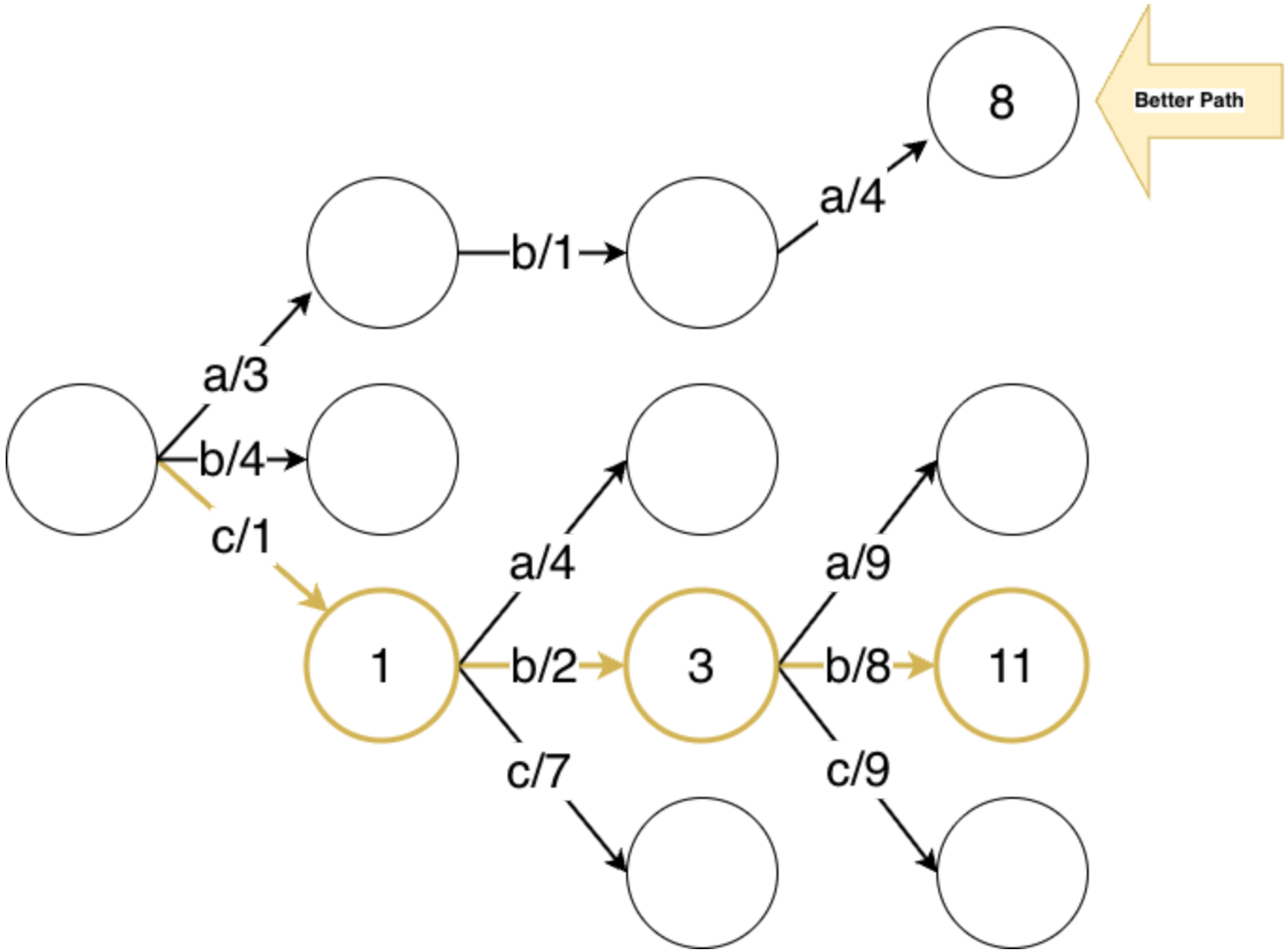
Graph Shortest Path: Greedy decoding (1)

- **Goal:** find the best (lowest-scoring) path in the graph



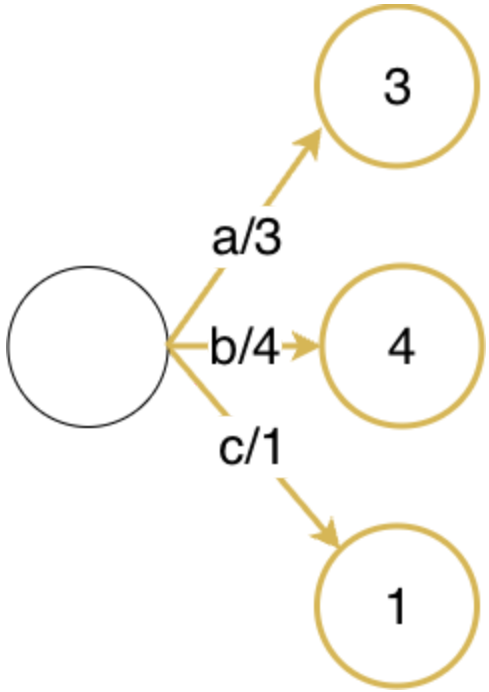
Graph Shortest Path: Greedy decoding (2)

Goal: find the best (lowest-scoring) path in the graph



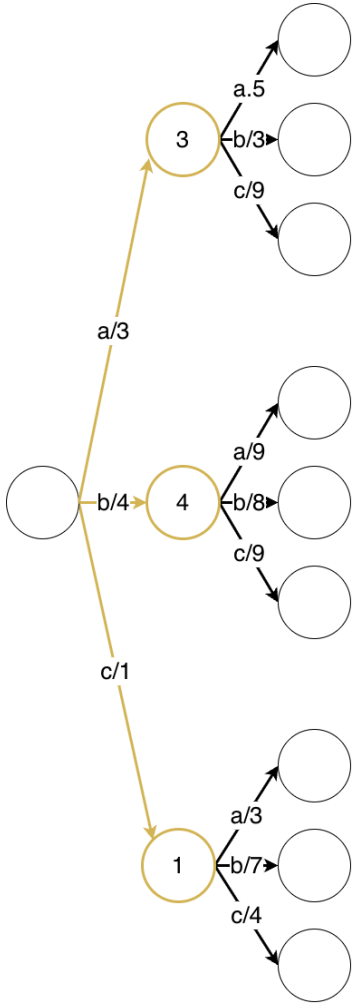
Graph Shortest Path: Beam Search (1)

- **Algorithm — repeat:**
  i. Extend current candidates by all possibilities
  ii. Sort by score and keep the N best



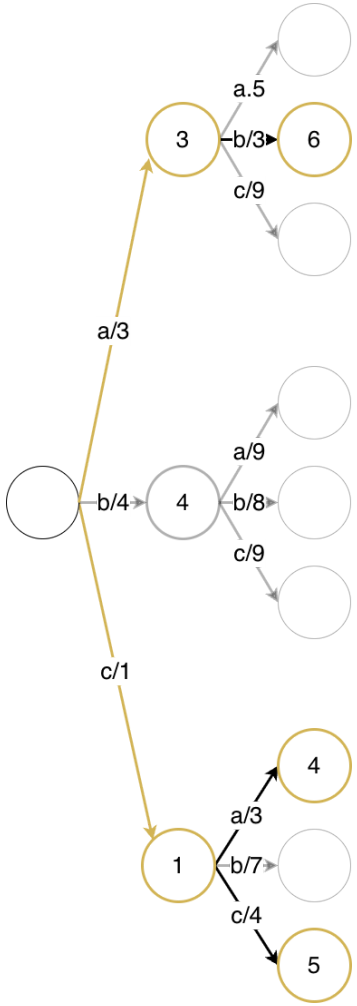
Graph Shortest Path: Beam Search (2)

$N = 3$



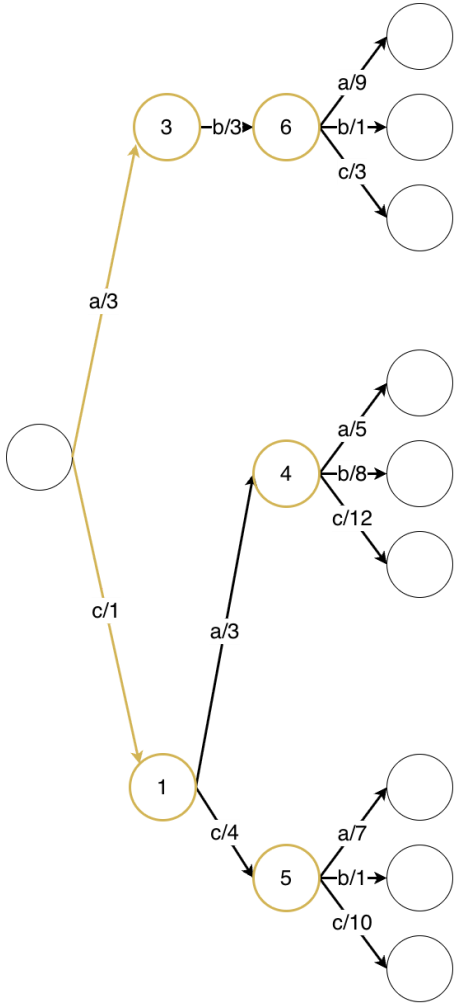
Graph Shortest Path: Beam Search (3)

$N = 3$

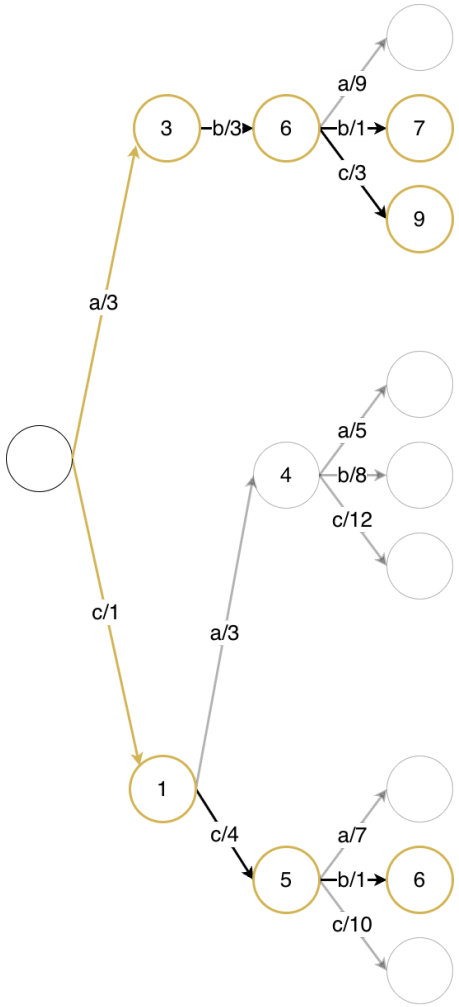


Graph Shortest Path: Beam Search (4)

$N = 3$



Graph Shortest Path: Beam Search (5)



Inference

- **Goal:** find the best Y (transcription) given an X (speech)
- Use beam search to find:

$$Y^* \approx \arg\max_Y \log P(Y \mid X) + \log P(Y)$$

```
Return N-best list:
[c, c, b], score = 6
[a, b, b], score = 7
[a, b, c], score = 9
```

How does the LM enter the Decoder?

- Fundamental equation in the log domain, with two tunable knobs:

$$Y^* = \arg\max_Y \; \log P_{AM}(Y \mid X) + \lambda \log P_{LM}(Y) + \beta |Y|$$

- λ **LM weight**: how much we trust the LM relative to the acoustic model
- β **length (insertion) bonus**: every extra word multiplies in another probability < 1, so without it the LM favors short outputs
- λ and β are tuned on a **development set** (grid or random search), never on the test set

λ too small

The output is acoustically plausible but ungrammatical.

λ too large

The system ignores the audio and produces fluent but wrong text, and rare words and names disappear.

Shallow Fusion: the LM inside the Beam

- **Shallow fusion** applies the LM *during* beam search
- At every step each hypothesis is extended by a candidate token
- The new score adds the acoustic and the LM contribution:
  - $\text{score}(h') = \text{score}(h) + \log P_{AM}(\text{tok} \mid \ldots) + \lambda \log P_{LM}$
- Then keep only the best N hypotheses (the beam)
- The LM therefore **changes which hypotheses are pruned**, so it can rescue a path that the acoustics alone would have discarded

Illustrative scores (log-probabilities, made-up numbers):

Hypothesis	AM	LM	AM + λ LM ($\lambda = 1$)
wreck a nice beach	-10.1	-21.0	-31.1
recognize speech	-10.8	-14.2	**-25.0**

- Acoustically the first hypothesis is slightly better
- With the LM, the second one wins

N-best Rescoring: the LM after the Search

1. **First pass**: decode with beam search and keep the **N-best list** (we already produce it, see the beam search slides)
2. **Second pass**: score every candidate with a (bigger, slower) LM and combine the scores
3. Pick the best candidate according to the combined score

- The rescoring LM can be anything: a large n-gram, an LSTM or Transformer LM, even an LLM
- It can look at the **complete** hypothesis, so it is not restricted to left context

	Shallow fusion	N-best rescoring
When	during the beam search	after the search
LM sees	partial hypothesis	whole hypothesis
Can rescue pruned paths	yes	no (only what is in the list)
LM size / cost	limited, called often	can be large, called N times
Typical use	streaming, first pass	offline, second pass

Key Takeaways: Language Models and Decoding

- ASR is a search problem: $\hat{w} = \arg\max_w P(o \mid w)\, P(w)$, an **acoustic model** combined with a **language model**
- **N-gram LMs** (Markov assumption + counts + smoothing) are cheap and still widely used, **perplexity** measures their quality
- **Greedy decoding** takes the best choice at every step and can get stuck, **beam search** keeps the best N partial hypotheses and returns an N-best list
- The LM enters via **shallow fusion** (inside the beam) or **rescoring** (on the N-best list), controlled by the weight λ and the length bonus β
- The same decoding view carries over to neural ASR, as we will see for CTC

Deep Learning for Automatic Speech Recognition ~2015 - Today

The Big Picture: Three Paradigms of End-to-End ASR

- Classic ASR glued together separate acoustic model, lexicon, and language model (recall the *alphabet soup*)
- **End-to-end** ASR replaces the whole pipeline with **one neural network** mapping audio $\rightarrow$ text
- The core difficulty is always the same: the audio has T frames, the text has U tokens, and $T \neq U$ with no given alignment
- Three families of models solve this alignment problem differently — **almost every modern system is one of these three (or a hybrid):**

The Big Picture: Three Paradigms of End-to-End ASR

⊙ CTC

Connectionist Temporal Classification. Predicts one label *per frame* (incl. a `blank`), then collapses repeats/blanks. Assumes outputs are **conditionally independent** given the audio → no internal language model. Fast, simple, non-autoregressive.

⊙ RNN-T / Transducer

Recurrent Neural Network Transducer. Combines an audio encoder with a label **prediction network** (an internal LM) via a **joint network**. Emits tokens *frame-synchronously* → **natively streaming**, the production workhorse.

⊙ AED

Attention-based Encoder-Decoder. Encoder reads all audio, decoder **attends** and emits tokens autoregressively ($P(y_i \mid y_{<i}, X)$). Strong implicit LM, but needs the **whole utterance** → not natively streaming.

	Streaming?	Internal LM?	Autoregressive?
CTC	yes	no	no
AED	no	yes	yes
RNN-T	yes	yes	yes

- **Map to systems in this lecture:** Deep Speech = **CTC** exemplar (next) · Whisper = **AED** exemplar (later in this lecture) · RNN-T = the **streaming** workhorse (after LAS)

Output Units: Why Modern ASR Decodes Subwords

- All three paradigms map audio $\rightarrow$ a sequence of **tokens** — but *what is a token?*
- **Characters** (Deep Speech): tiny vocabulary, but long target sequences U and the model must learn spelling from scratch
- **Whole words**: short sequences, but a huge, open vocabulary — every unseen word is unrecognizable (OOV)
- **Subwords** (the modern default): data-driven units in between — frequent words stay whole, rare words split into pieces

Subword tokenization

BPE / WordPiece / SentencePiece learn a fixed vocabulary of subword units from training text. Open-vocabulary (no OOV), shorter targets U than characters, and no hand-built lexicon.

- Why it matters for ASR: shorter $U \rightarrow$ cheaper RNN-T lattices & AED decoding, and an internal LM over *meaningful* units
- **Whisper, modern RNN-T transducers (Parakeet / Canary), and most AED systems all decode subwords** — characters are now mostly a teaching baseline

Technische Hochschule Rosenheim

THE CTC LOSS

Forced Alignment: the Idea behind CTC Training

⊙ Forced Alignment

Given audio **and** its known transcript, find **which audio frames correspond to which tokens** in time.

It is *forced* because the token sequence is fixed: we only solve the alignment, not the recognition.

- Classical systems needed such alignments (found with Viterbi) as frame-level labels to train acoustic models
- End-to-end models have **no given alignment**: the audio has T frames, the text has U tokens, $T \neq U$
- **CTC** solves this by summing over *all* possible alignments with dynamic programming
- To build intuition, we first align a tiny example, the word "CAT"

Forced Alignment Example: Aligning the word "CAT"

Naive Approach: expanding all possible allignments, choosing the best one (most likely)



Image source: https://research.nvidia.com/labs/conv-ai/blogs/2023/2023-08-forced-alignment/

Continuation of the Example: Aligning the word "CAT"

Technische Hochschule Rosenheim

- Improving by spotting graph redundancies
- don't need to calculate all cumulative products for every path
- no need to draw the full tree graph
- Examining the two 'A' nodes at 't=3', shows that the node with lower cumulative product, and its downstream nodes can be discarded



Continuation of the Example: Aligning the word "CAT"

- These redundancies exist between any nodes with the same (s, t) values
- all their downstream nodes will have the same structure
- only necessary to keep the node that corresponds to the most probable path from the start node to (s,t).

So, for each (s, t) only the most probable path to (s,t) needs to be kept

- all the non-winning nodes can be discarded (inkl. its downstream nodes) $\rightarrow$ less computations

Continuation of the Example: Aligning the word "CAT"



Image source: https://research.nvidia.com/labs/conv-ai/blogs/2023/2023-08-forced-alignment/

Summary forced Alignments:

- creating the Viterbi matrix $S \times T$ (tokens x time steps)
- every element corresponds to a node in the graph
- filling it column by column and row by row
- all possible alignments start with the first token $s = 1$ at $t = 1$, rest as 0
- for each token s at time t:
  - calculate the cumulative probability of the node
  - compare it to the cumulative probabilities of the nodes with the same (s, t) values
  - keep the most probable path to (s, t)
  - discard the rest

	t=1	t=2	t=3	t=4	t=5
s=0 AKA token is C	??	??	??	??	??
s=1 AKA token is A	??	??	??	??	??
s=2 AKA token is T	??	??	??	??	??

The CTC Loss

- **Goal:**
  - i. Input speech $X = [x_1, \ldots, x_T]$
  - ii. Output transcription $Y = [y_1, \ldots, y_U]$
- **Compute:**

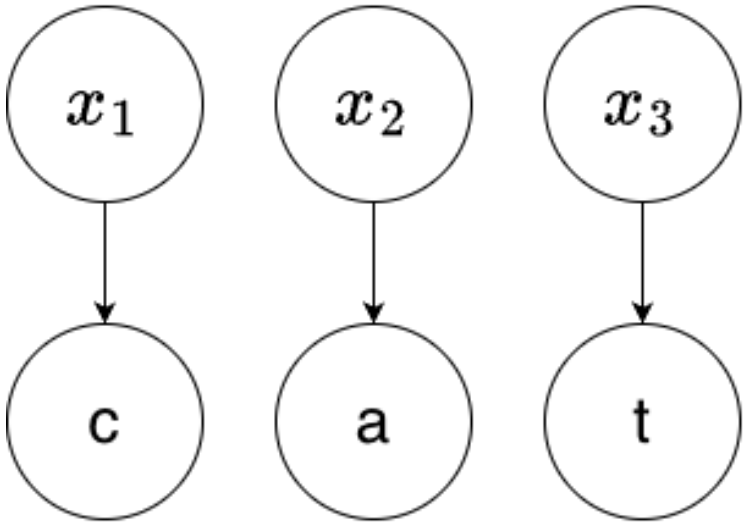
$$\log P(Y \mid X; \Theta)$$

> The loss needs to be differentiable w.r.t. model parameters Θ

The CTC Loss

- **Example:**
  - i. Input speech $X = [x_1, x_2, x_3]$
  - ii. Output transcription $Y = [c, a, t]$
- **Compute:**

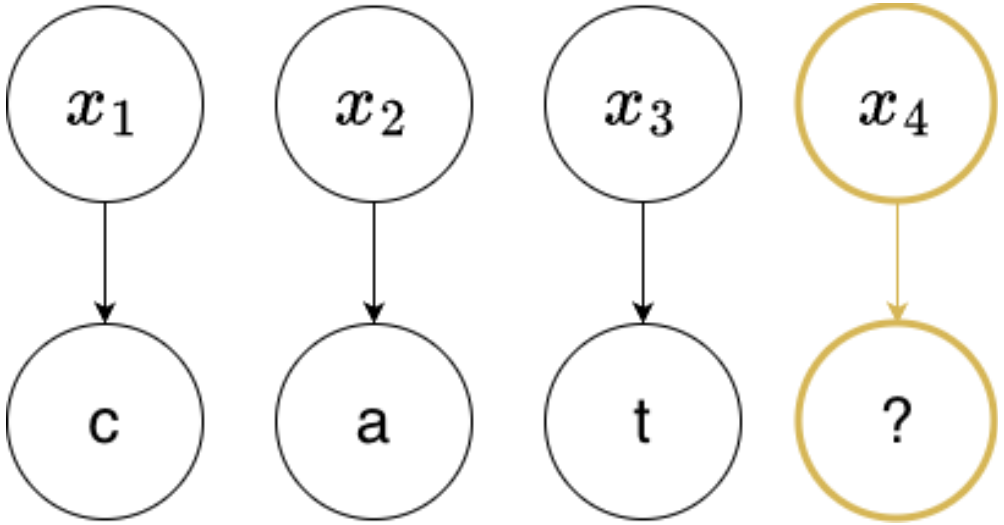
$$\log P(c \mid x_1) + \log P(a \mid x_2) + \log P(t \mid x_3)$$



The CTC Loss

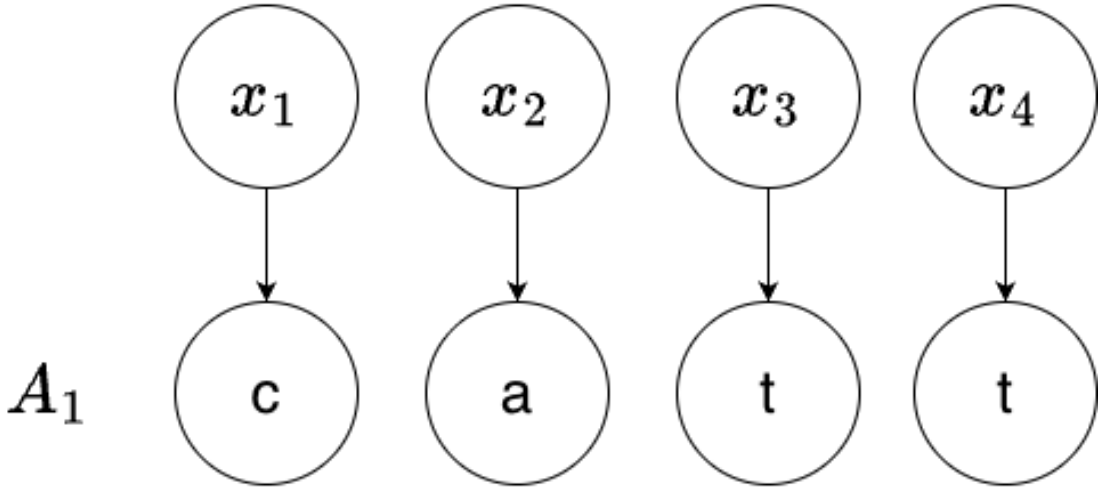
- **Example:**
  - i. Input speech $X = [x_1, x_2, x_3, \mathbf{x_4}]$
  - ii. Output transcription $Y = [c, a, t]$
- **Compute:**

$$\log P(c \mid x_1) + \log P(a \mid x_2) + \log P(t \mid x_3) + \boxed{\log P(? \mid x_4)}$$



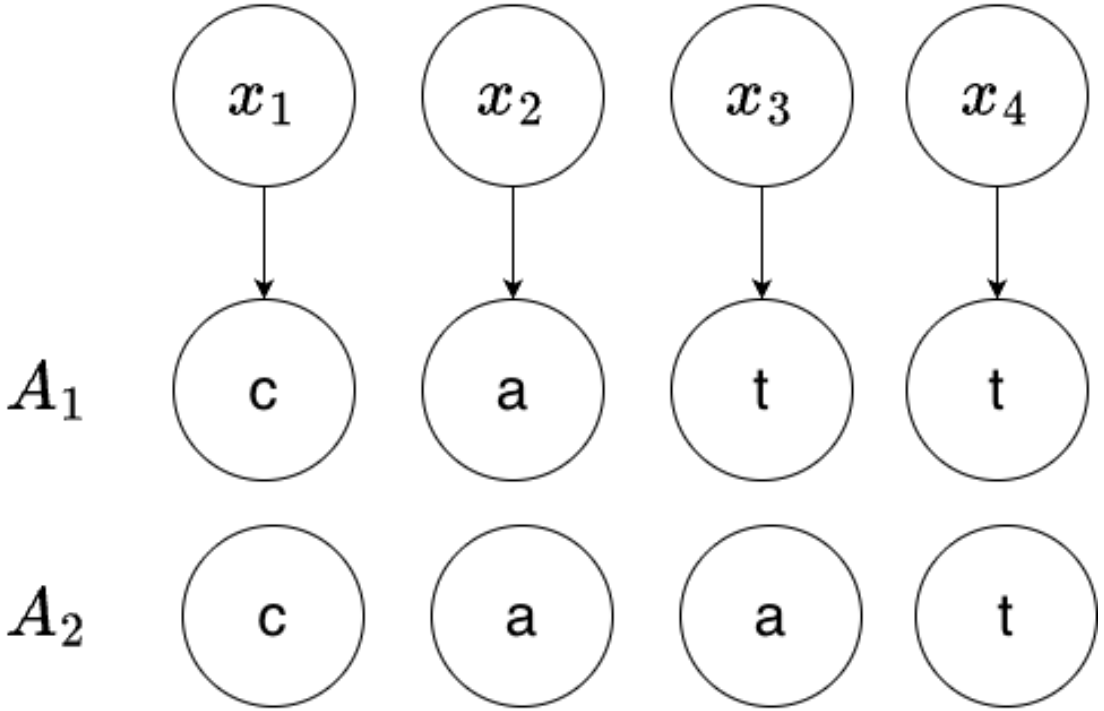
The CTC Loss

- **Alignment:** one or more of each input maps to an output



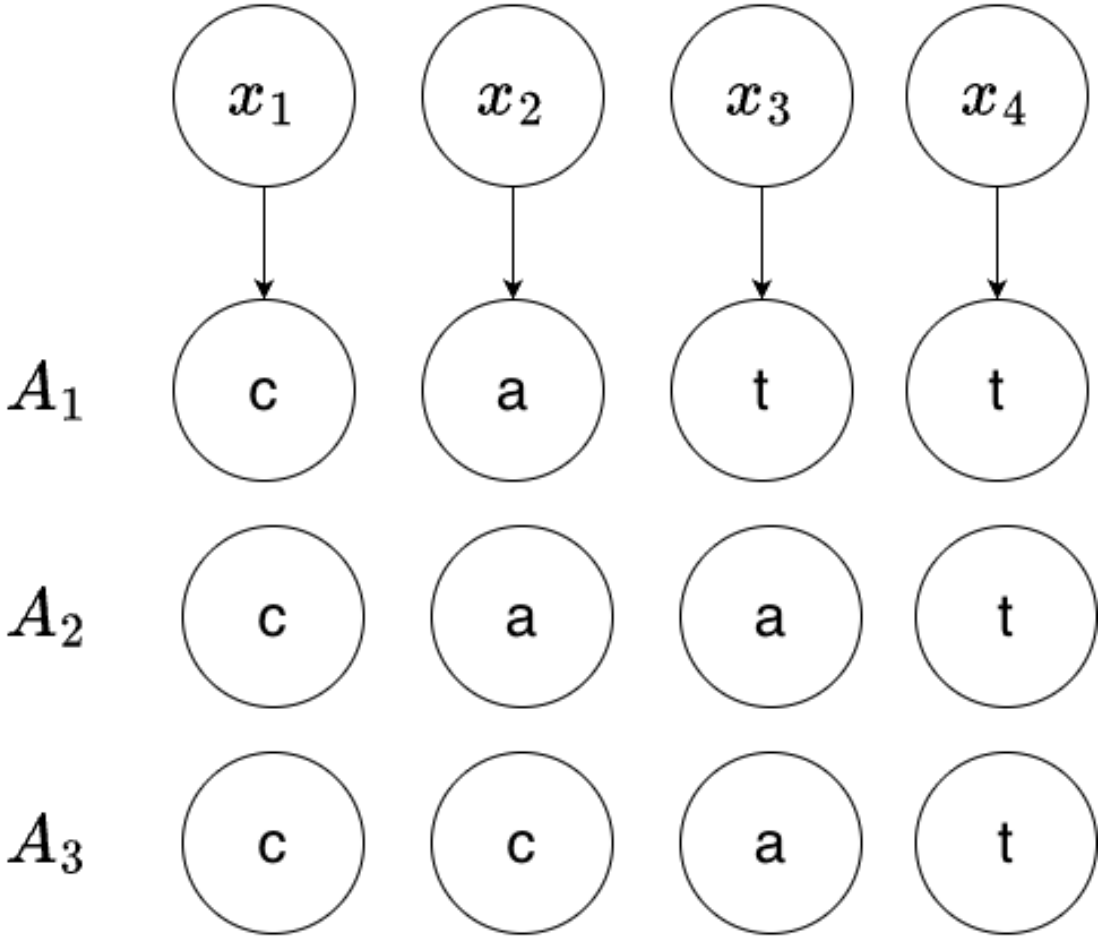
The CTC Loss

- **Alignment:** one or more of each input maps to an output



The CTC Loss

- **Alignment:** one or more of each input maps to an output



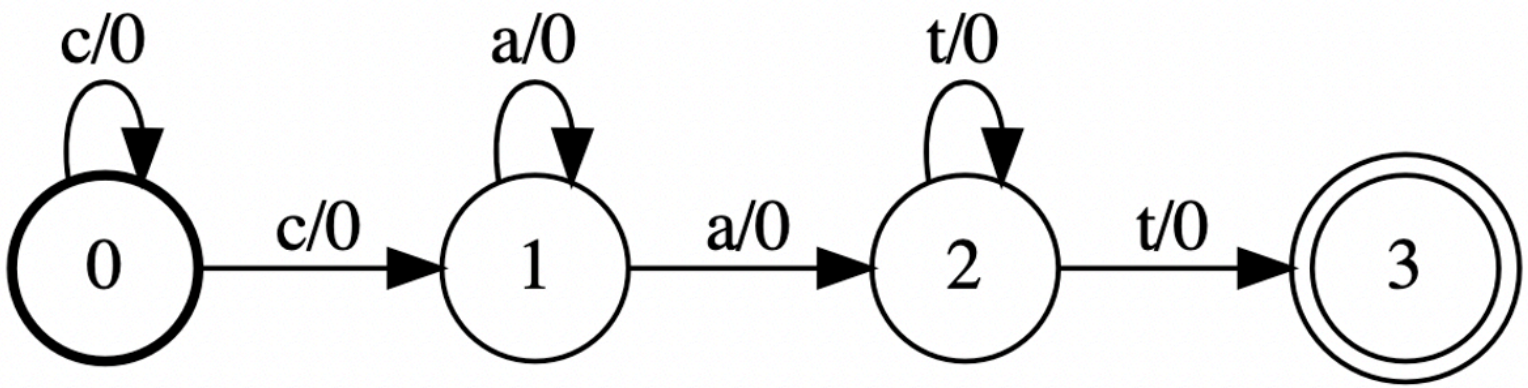
The CTC Loss

- **Which alignment should we use to compute the score?**
- **All of them!**

$$\log P(Y \mid X) = \log \left[P(A_1 \mid X) + P(A_2 \mid X) + P(A_3 \mid X) \right]$$
$$= \text{logsumexp} \left[\log P(A_1 \mid X),\ \log P(A_2 \mid X),\ \log P(A_3 \mid X) \right]$$

The CTC Loss

- **Alignment graph for $Y = [c, a, t]$**



The CTC Loss

- **Problem:** X has T frames and Y has U characters
- If $T = 1000$ and $U = 100 \sim 6.4 \times 10^{138}$ alignments
- $\binom{T-1}{U-1}$ combinations (combinatorics exercise, "stars and bars")
- Compute the number of possible alignments:

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

The CTC Loss (1)

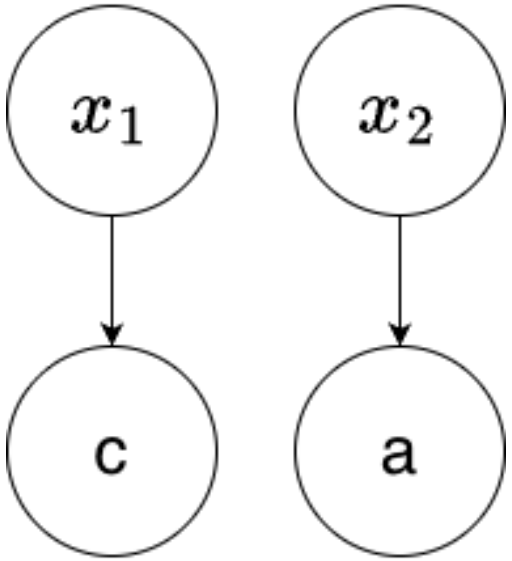
- **Solution: the forward algorithm (dynamic programming)**

- Forward variable α_t^u as the score for all alignments
  - of length t
  - which end in y_u
- e.g., α_3^a for $Y = [c, a, t]$ and $X = [x_1, x_2, x_3, x_4]$:
  - is the score for all alignments of length three
  - which end in a

The CTC Loss (2)

- **Solution:** the forward algorithm (dynamic programming)
- Example: $X = [x_1, x_2, x_3, x_4], Y = [c, a, t]$

$$\alpha_2^a = \log P(c \mid x_1) + \log P(a \mid x_2)$$



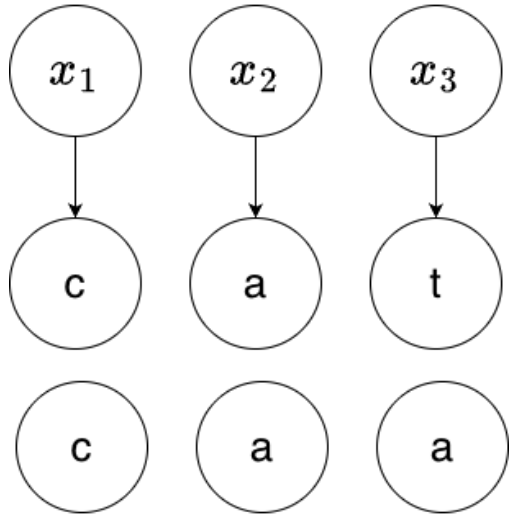
The CTC Loss (3)

- **Solution:** the forward algorithm (dynamic programming)
- Example: $X = [x_1, x_2, x_3, x_4], Y = [c, a, t]$

$$\alpha_3^a = \text{logsumexp}\left[\log P(A_1),\ \log P(A_2)\right]$$

$$\log P(A_1) = \log P(c \mid x_1) + \log P(c \mid x_2) + \log P(a \mid x_3)$$

$$\log P(A_2) = \log P(c \mid x_1) + \log P(a \mid x_2) + \log P(a \mid x_3)$$



The CTC Loss (4)

- **Solution:** the forward algorithm (dynamic programming)
- Example: $X = [x_1, x_2, x_3, x_4], Y = [c, a, t]$

$$\alpha_3^a = \text{logsumexp}\left[\log P(A_1),\ \log P(A_2)\right]$$

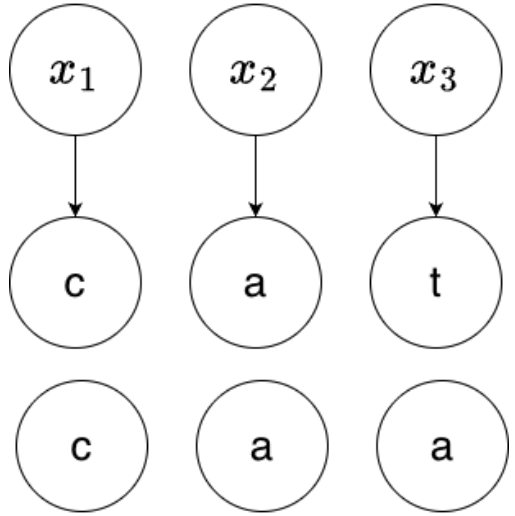
$$\log P(A_1) = \boxed{\log P(c \mid x_1) + \log P(c \mid x_2)} + \log P(a \mid x_3)$$

- $\rightarrow \alpha_2^c$

$$\log P(A_2) = \boxed{\log P(c \mid x_1) + \log P(a \mid x_2)} + \log P(a \mid x_3)$$

- $\rightarrow \alpha_2^a$

after time step 2 probability for c or a



The CTC Loss (5)

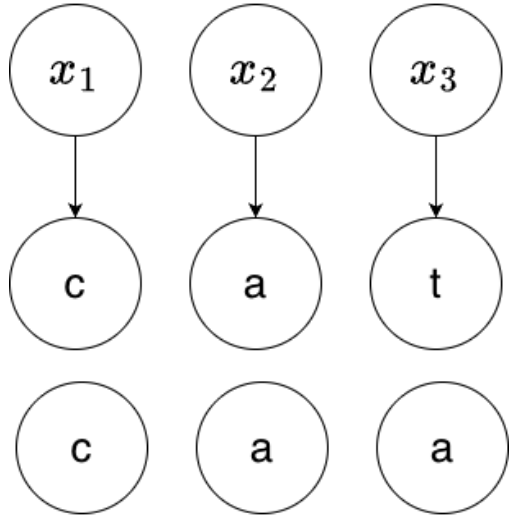
- **Solution:** the forward algorithm (dynamic programming)
- Example: $X = [x_1, x_2, x_3, x_4], Y = [c, a, t]$

$$\alpha_3^a = \text{logsumexp}\left[\log P(A_1),\ \log P(A_2)\right]$$

$$= \text{logsumexp}\left(\left[\alpha_2^c,\ \alpha_2^a\right] + \log P(a \mid x_3)\right)$$

$$\log P(A_1) = \alpha_2^c + \log P(a \mid x_3)$$

$$\log P(A_2) = \alpha_2^a + \log P(a \mid x_3)$$



The CTC Loss, simplified version (6)

- **Solution:** the forward algorithm (dynamic programming)

$$X = [x_1, x_2, \ldots, x_T], \qquad Y = [y_1, y_2, \ldots, y_U]$$

Initialization ($t = 1$):

$$\alpha_1^1 = \log P(y_1 \mid x_1), \qquad \alpha_1^u = -\infty \quad (u > 1)$$

Recursion (shown explicitly):

$$\alpha_t^u = \log\left(e^{\alpha_{t-1}^u} + e^{\alpha_{t-1}^{u-1}}\right) + \log P(y_u \mid x_t)$$

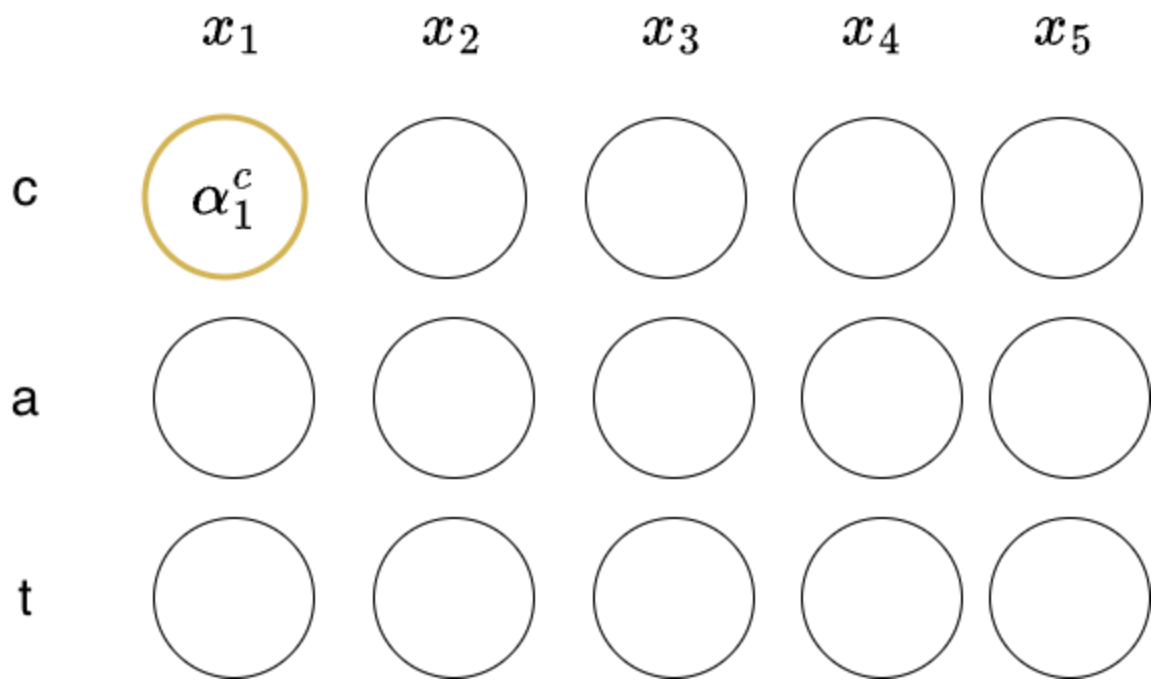
where $\mathrm{logsumexp}[z_1, \ldots, z_n] := \log \sum_i e^{z_i}$ (the smooth max) used as shorthand from here on.

Final score: log-probability of all alignments, summed:

$$\log P(Y \mid X) = \log \sum_{A \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^{T} P(a_t \mid x_t) = \alpha_T^U$$

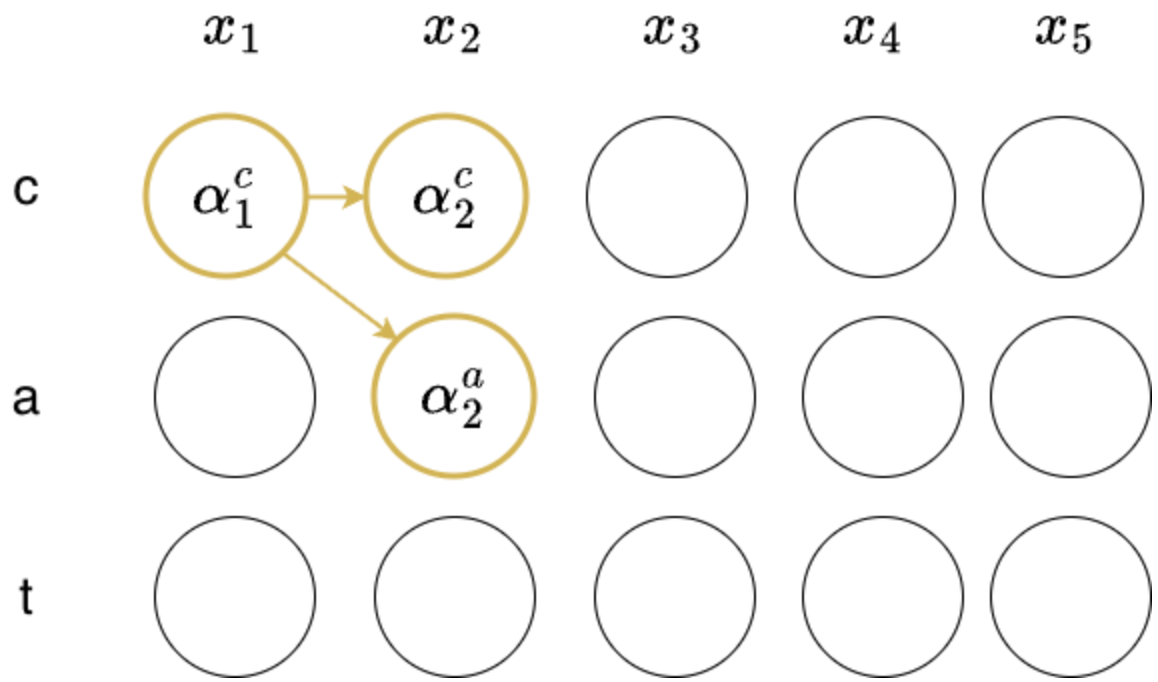
The CTC Loss Example (1)

- **Solution:** the forward algorithm (dynamic programming)



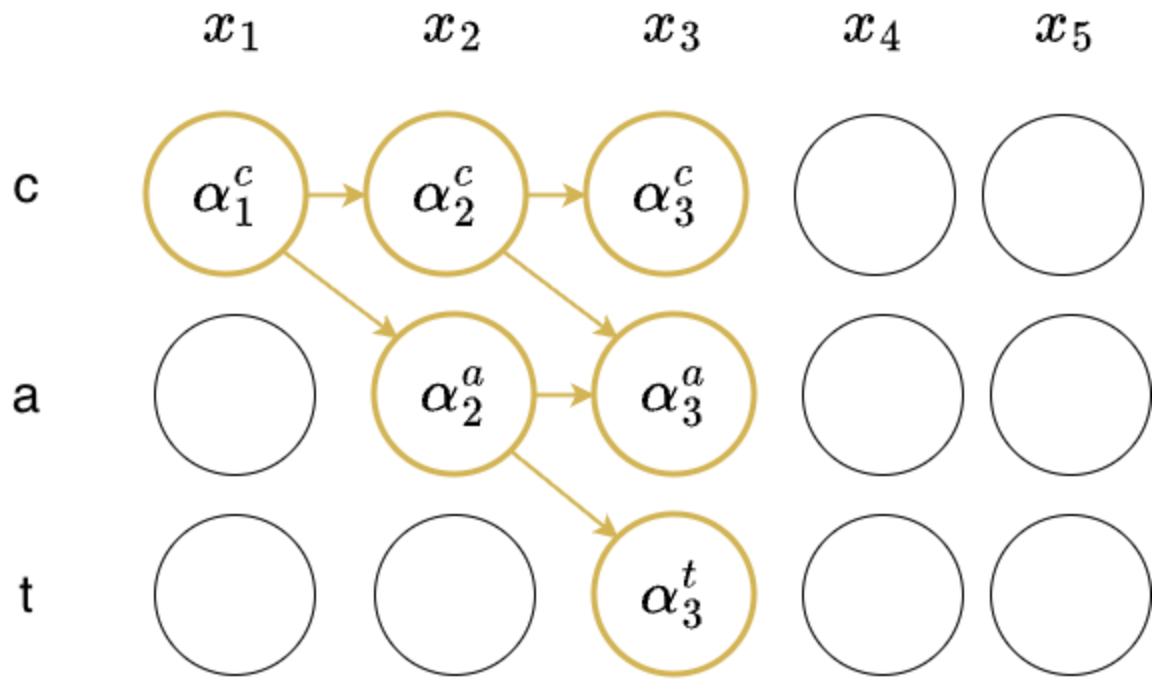
The CTC Loss Example (2)

- **Solution:** the forward algorithm (dynamic programming)



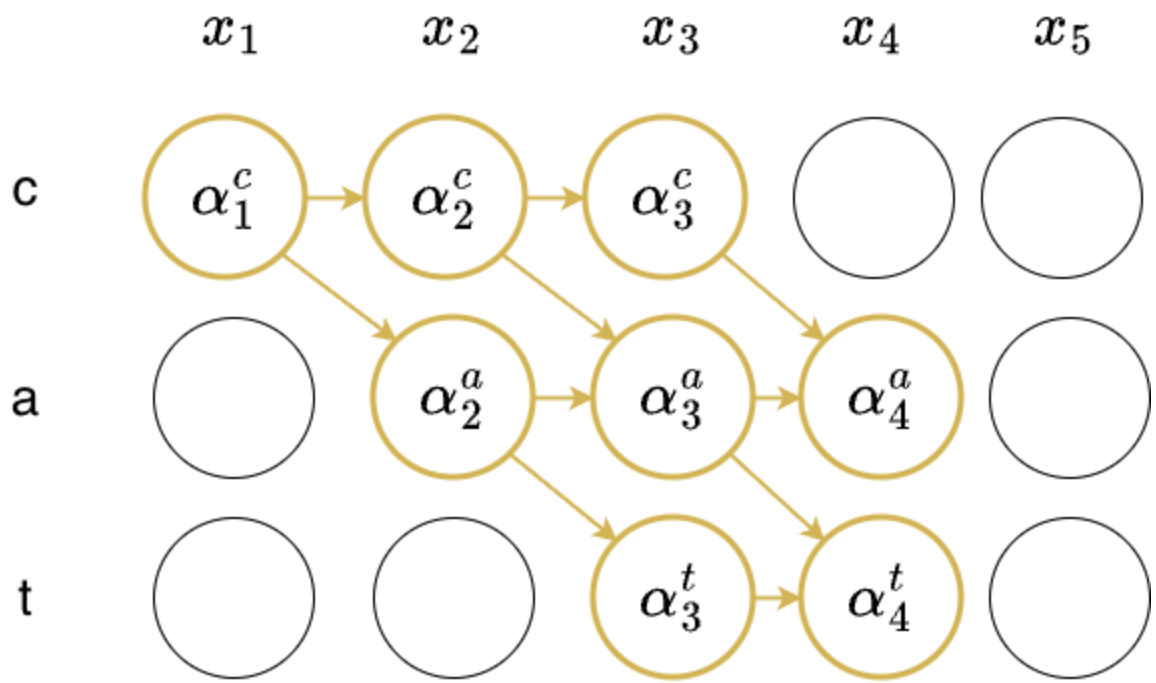
The CTC Loss Example (3)

- **Solution:** the forward algorithm (dynamic programming)



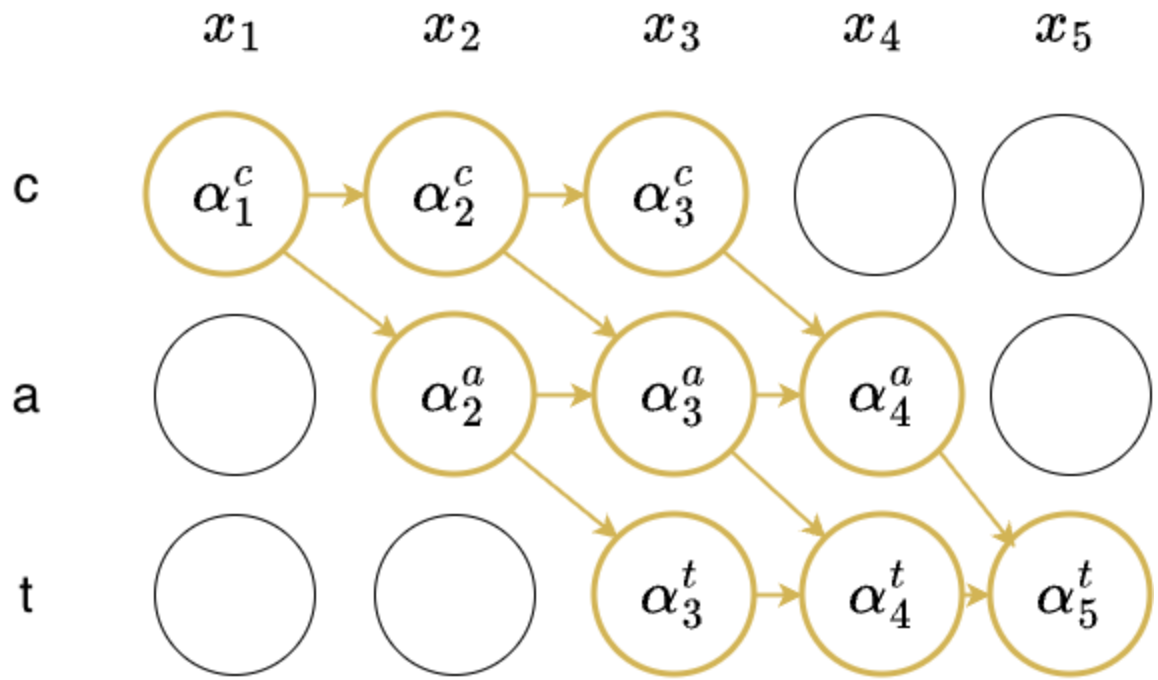
The CTC Loss Example 4

- **Solution:** the forward algorithm (dynamic programming)



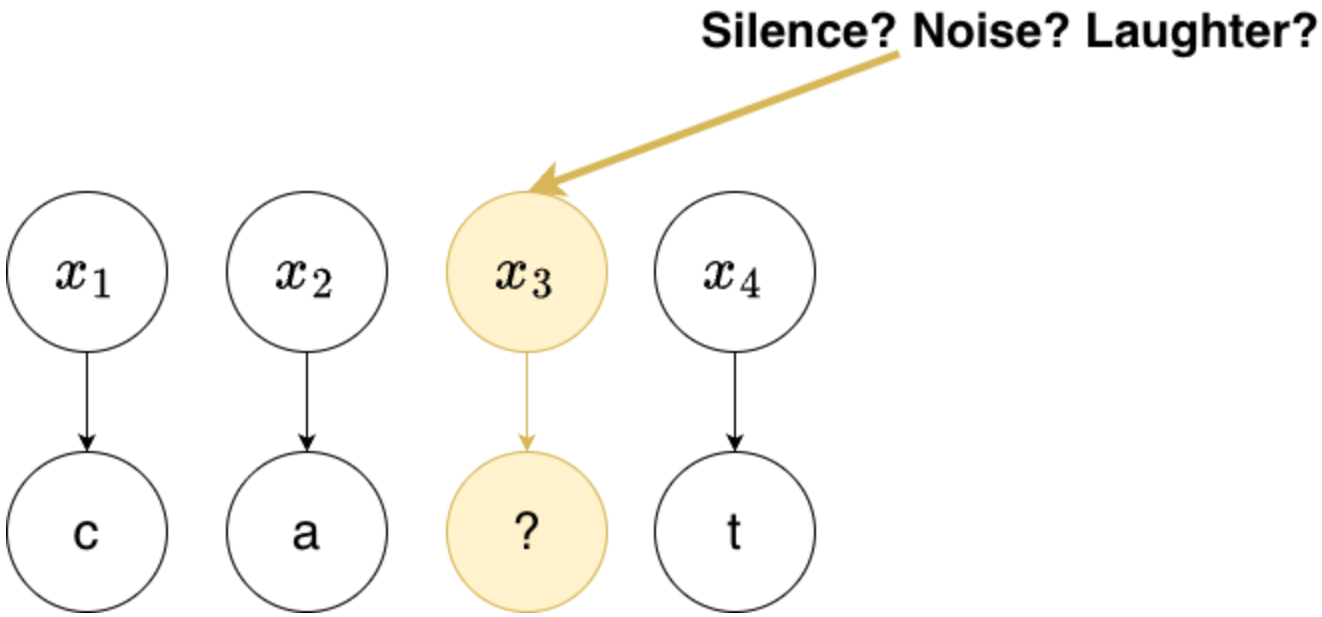
The CTC Loss Example 5

- **Solution:** the forward algorithm (dynamic programming)



The CTC Loss Update (1)

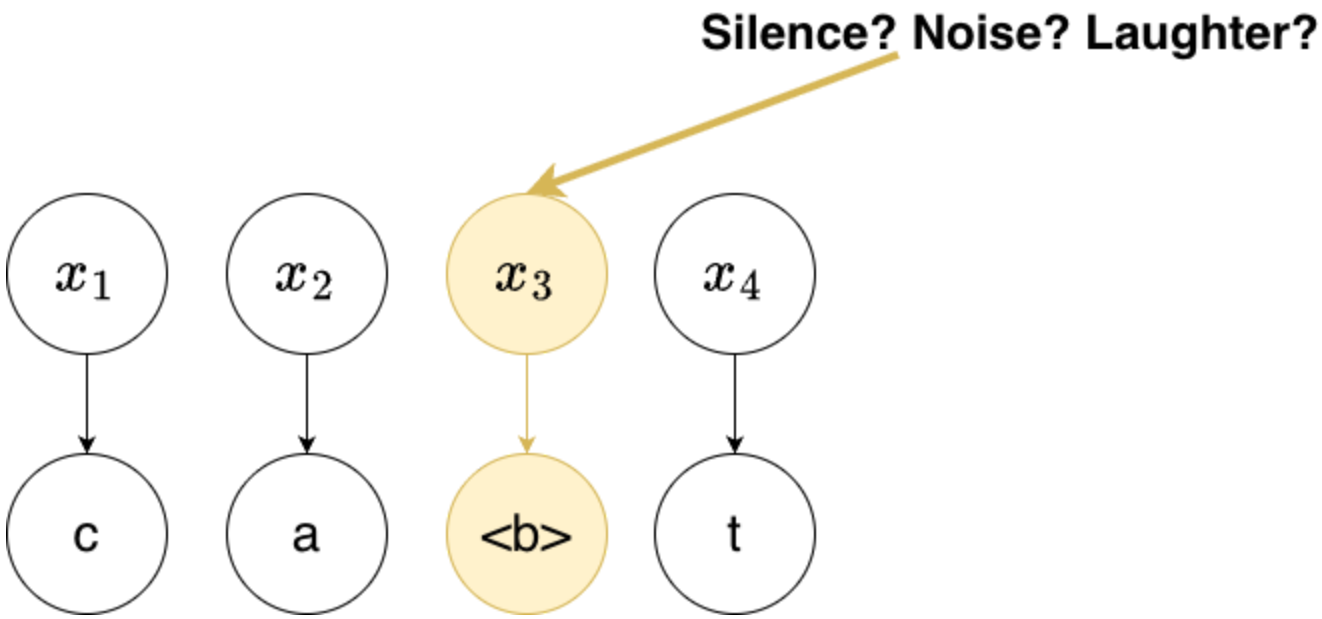
Problem: not every input corresponds to "speech"



The CTC Loss Update (2)

Solution: use a "garbage" or blank token: `<b>`

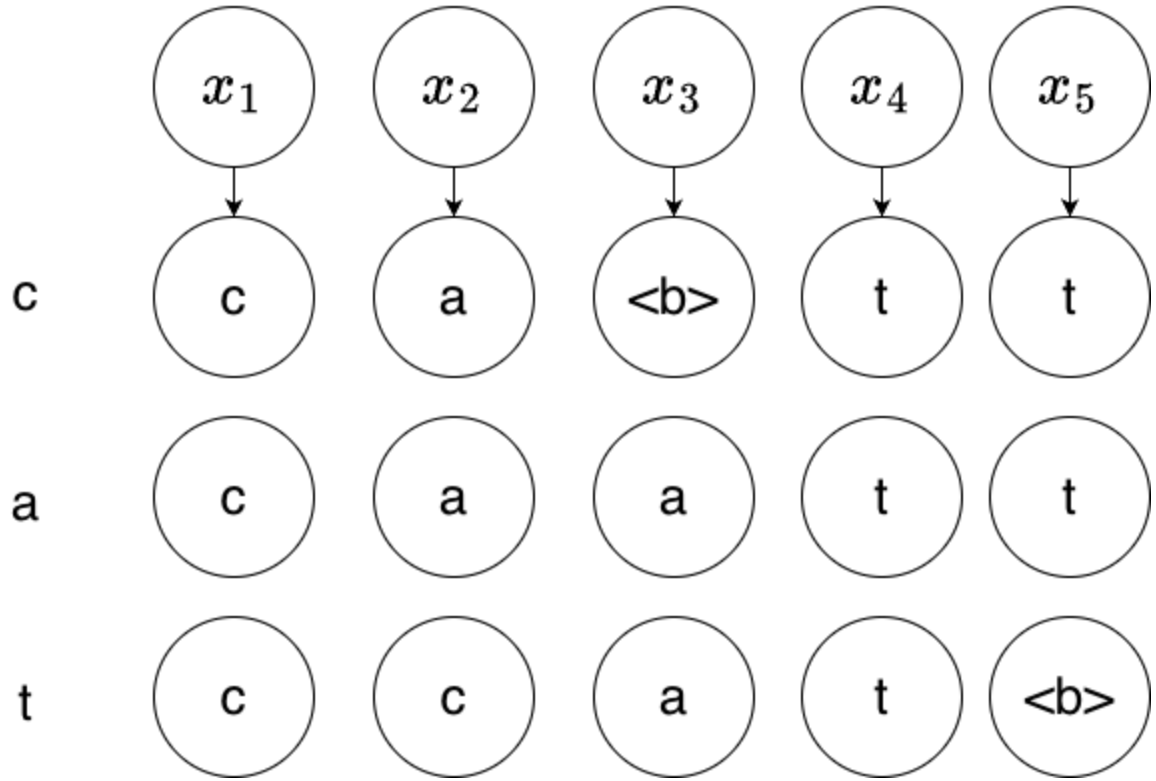
Make the blank token optional



The CTC Loss Update (3)

Solution: use a "garbage" or blank token: `<b>`

Make the blank token optional

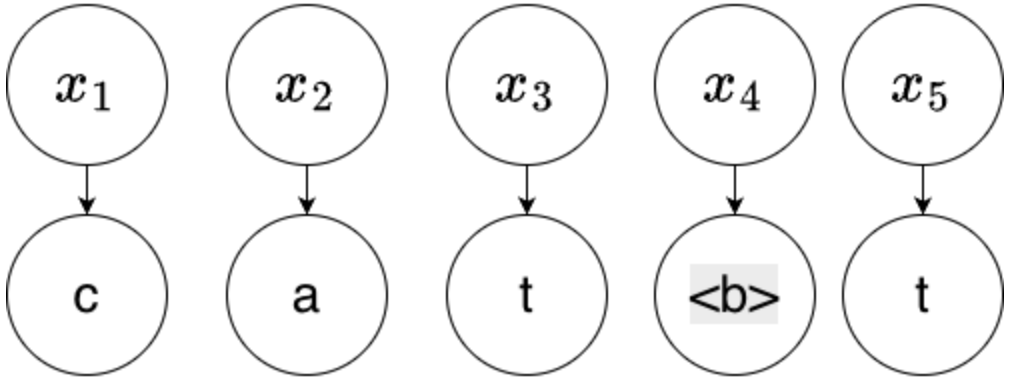


The CTC Allowed Alignments (1)

Solution: use a "garbage" or blank token: `<b>`

Make the blank token optional

Is this alignment allowed?



> No! The decoded sequence would be 'catt'

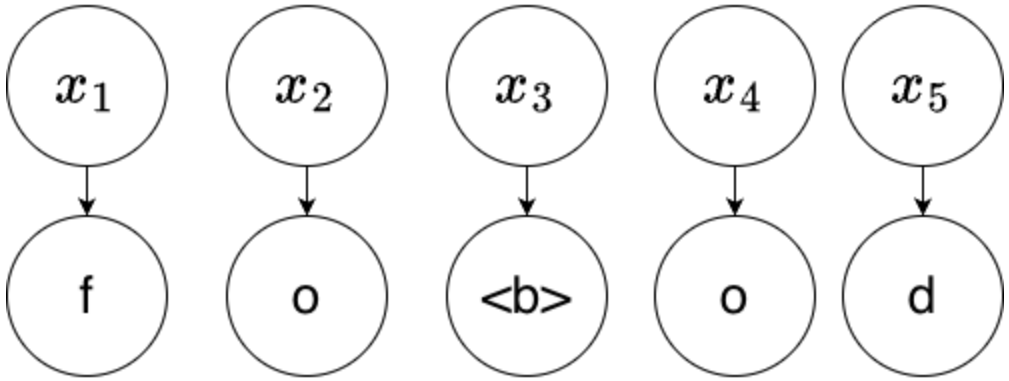
The CTC Allowed Alignments (2)

Solution: use a "garbage" or blank token: **<b>**

Blank token is optional

$\rightarrow$ **except between repeats in** Y

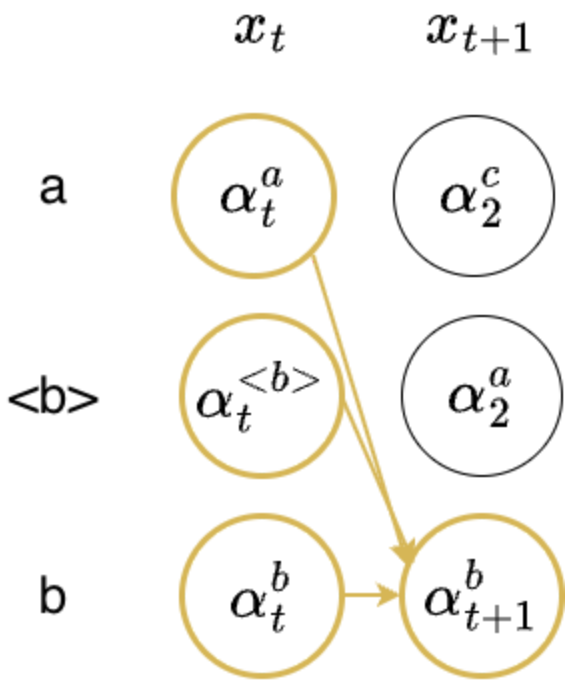
$Y = [f, o, o, d]$



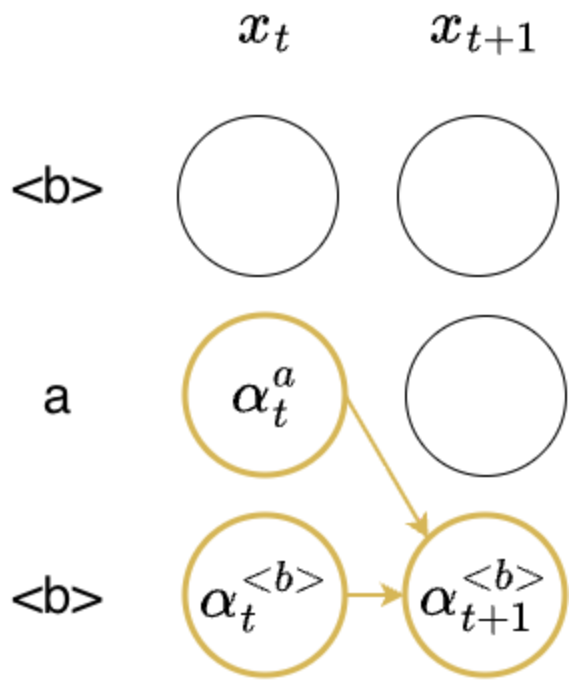
The CTC Loss Recursion with Blanks

CTC recursion: three cases

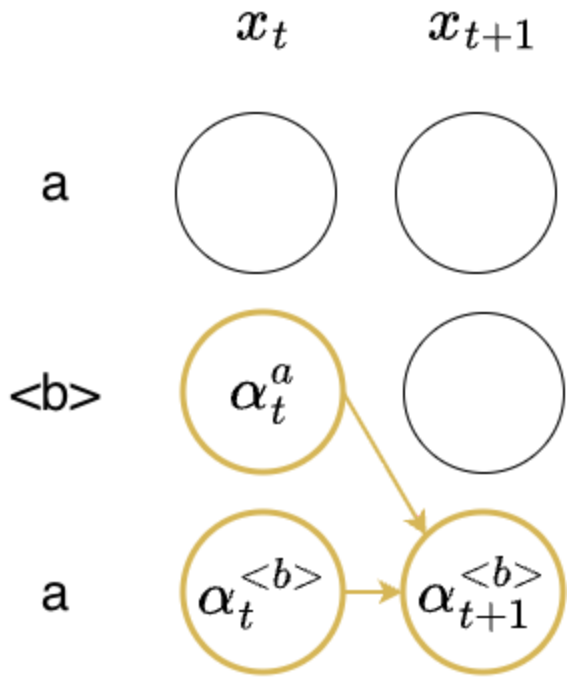
Case 1: blank is optional



Case 2: output is not optional



Case 3: repeats, blank is not optional



The CTC Loss - Full Version (1)

Original input and target:

$$X = [x_1, x_2, \ldots, x_T], \qquad Y = [y_1, y_2, \ldots, y_U]$$

Introduce the **blank** symbol $\varnothing$ and define the expanded target sequence

$Y' = [\varnothing, y_1, \varnothing, y_2, \ldots, \varnothing, y_U, \varnothing]$ with elements $Y' = [y'_1, y'_2, \ldots, y'_{2U+1}]$.

The forward variable now indexes positions in Y':

$$\alpha_t^s \quad \text{for } s = 1, \ldots, 2U+1$$

Initialization ($t = 1$): $\alpha_1^1 = \log P(y'_1 \mid x_1), \qquad \alpha_1^2 = \log P(y'_2 \mid x_1), \qquad \alpha_1^s = -\infty \quad (s > 2)$

with boundary values $\alpha_t^0 = \alpha_t^{-1} = -\infty$.

The CTC Loss - Full Version (2)

Recursion:

$$\alpha_t^s = \text{logsumexp}\Big(\alpha_{t-1}^s, \alpha_{t-1}^{s-1}, \alpha_{t-1}^{s-2} \text{ if } \text{skip}(s)\Big) + \log P(y'_s \mid x_t)$$

where the skip transition is allowed iff

$\text{skip}(s) \iff y'_s \neq \varnothing \ \wedge \ y'_s \neq y'_{s-2}$ and $\text{logsumexp}[z_1, \ldots, z_n] := \log \sum_i e^{z_i}$.

Final score: log-probability of all valid CTC alignments, summed:

$$\log P(Y \mid X) = \log \sum_{A \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^{T} P(a_t \mid x_t) = \text{logsumexp}(\alpha_T^{2U+1}, \alpha_T^{2U})$$

where $\mathcal{B}$ is the CTC collapse function: merge consecutive repeats, then remove blanks.

CTC Decoding (1): Greedy / Best Path

- At inference we have the per-frame distribution $P(a_t \mid x_t)$ over tokens plus `<b>`
- **Greedy (best path) decoding**:
  i. take the $\arg\max$ token in every frame
  ii. **merge repeated** tokens
  iii. **remove blanks**

```
frame argmax:  c c <b> a a <b> <b> t t
merge repeats: c <b> a <b> t
remove blanks: c a t
```

- Fast, no search, no LM needed
- But the **best path is not the best transcript**:
  - many different alignments collapse to the same text
  - the probability of a text is the **sum** over all of its alignments (exactly what the CTC loss computes)
  - one very likely alignment can lose against a text with many moderately likely alignments

Conditional independence

CTC predicts every frame independently given the audio, so it has **no internal language model**: spelling and grammar errors are not corrected by the network itself.

CTC Decoding (2): Prefix Beam Search with an LM

- Beam search over **prefixes** (the text so far), not over alignments
- For every prefix keep two probabilities:
  - p_b: all alignments of this prefix that end in a blank
  - p_{nb}: all alignments that end in a non-blank
- Different alignments that collapse to the **same prefix are merged by summing** their probabilities
- The beam keeps the N best prefixes after every frame

- **Adding the external LM**: whenever a prefix is extended by a new character or word, add the LM score:

$$\text{score} = \log P_{CTC}(\text{prefix}) + \lambda \log P_{LM}(\text{prefix}) + \beta\,|\text{prefix}|$$

- Same λ and β as before, tuned on a dev set
- Result: an **N-best list** that can be rescored with a larger LM
- Tools: `pyctcdecode` (KenLM n-gram, hotwords), `torchaudio` CTC decoder (flashlight), NeMo, Kaldi

Deep Speech

- Developed by Baidu Research.
- Uses a **Connectionist Temporal Classification (CTC) loss**.
- Employs **Recurrent Neural Networks (RNNs)** for acoustic modeling.
- Achieved state-of-the-art performance on several benchmarks.
- **End-to-end** architecture for speech recognition.

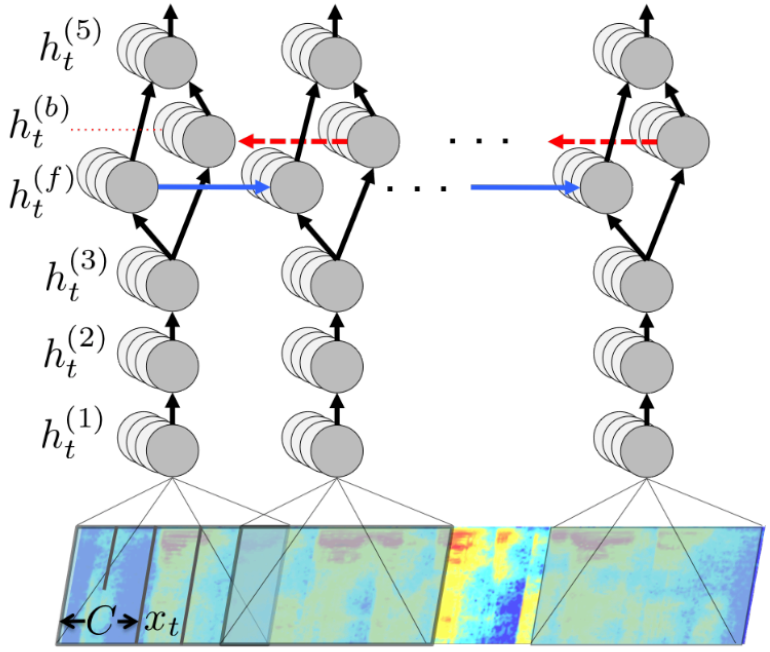


Figure 1: Structure of our RNN model and notation.

Image source: Hannun, A. et al., 2014. Deep Speech: Scaling up end-to-end speech recognition.

Deep Speech Architecture

- Input sequence: **Spectrogram of audio signal**
  - 80 linearly spaced frequency bins
  - 20ms frame size, 10ms frame-shift
- First Layers: Non-recurrent Feedforward layers for context modeling
- Training data is a set of X, Y pairs for each utternace
  - $X = \{x^{(1)}, y^{(1)}, \ldots, x^{(T)}, y^{(T)}\}$
- Frame at time t is processed by a stack of layers (l), including a context of C frames on each side

$$h_t^{(l)} = g(W^{(l)} h_t^{(l-1)} + b^{(l)})$$

- followed by a bi-directional RNNs, sequential processing of the frames
- concatenated result is then computed by the final layer to predict the output for each frame t
- softmax layer to predict the probability of each character in the alphabet, $\mathbb{P}(c_t|x)$
- Computing the loss function using the CTC loss

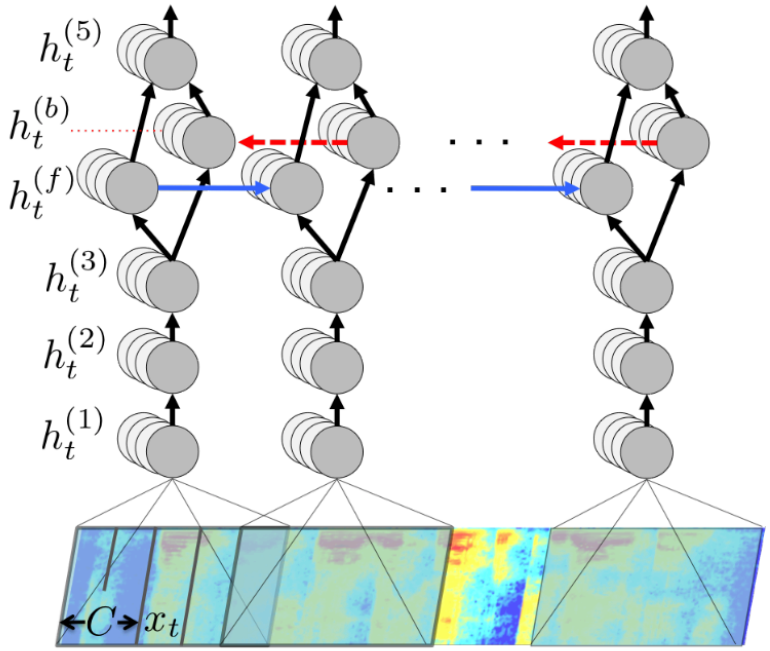


Figure 1: Structure of our RNN model and notation.

Image source: Hannun, A. et al., 2014. Deep Speech: Scaling up end-to-end speech recognition.

Deep Speech Results

Technische Hochschule Rosenheim

Model	SWB	CH	Full
Vesely et al. (GMM-HMM BMMI) [44]	18.6	33.0	25.8
Vesely et al. (DNN-HMM sMBR) [44]	12.6	24.1	18.4
Maas et al. (DNN-HMM SWB) [28]	14.6	26.3	20.5
Maas et al. (DNN-HMM FSH) [28]	16.0	23.7	19.9
Seide et al. (CD-DNN) [39]	16.1	n/a	n/a
Kingsbury et al. (DNN-HMM sMBR HF) [22]	13.3	n/a	n/a
Sainath et al. (CNN-HMM) [36]	11.5	n/a	n/a
Soltau et al. (MLP/CNN+I-Vector) [40]	**10.4**	n/a	n/a
Deep Speech SWB	20.0	31.8	25.9
Deep Speech SWB + FSH	12.6	**19.3**	**16.0**

Table 3: Published error rates (%WER) on Switchboard dataset splits. The columns labeled "SWB" and "CH" are respectively the easy and hard subsets of Hub5'00.

Key Takeaways: CTC

- **Problem**: T audio frames, U tokens, no alignment given
- **Idea**: add a **blank** token and define the probability of a transcript as the **sum over all valid alignments**
- The **forward algorithm** (dynamic programming over the $(2U + 1) \times T$ lattice) computes this sum efficiently and differentiably
- **Greedy decoding**: argmax, merge repeats, drop blanks
- **Prefix beam search** merges alignments of the same text and plugs in an **external LM** via λ and β
- Because of the conditional independence assumption, CTC models profit from an external LM, especially with little training data

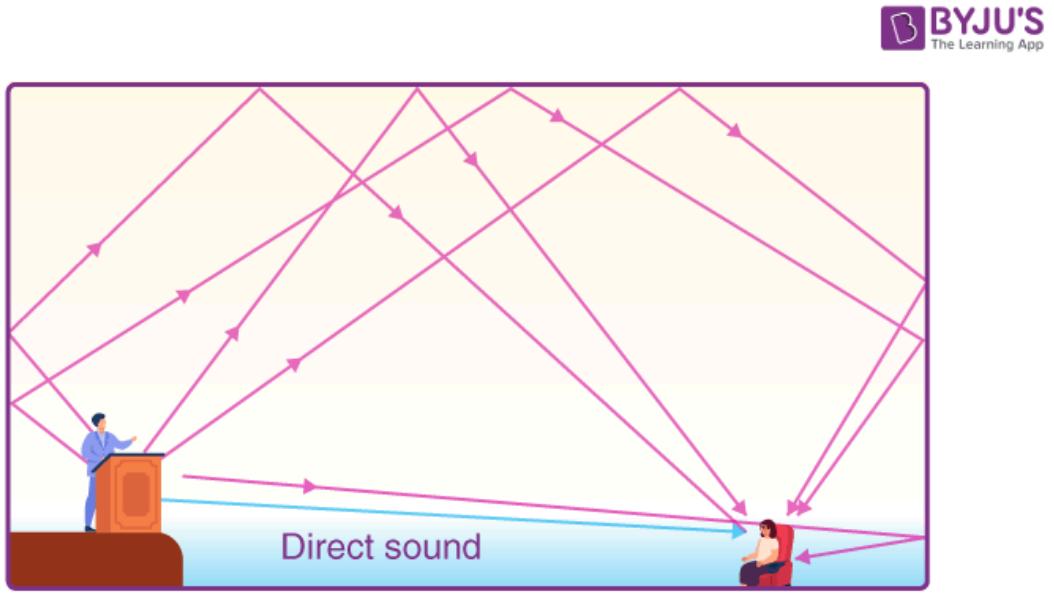
Data Augmentation

Reverberation

- **Reverberation** is the persistence of sound after the original sound is produced.
- occurs when sound waves reflect off surfaces such as walls, floors, and ceilings

The Process of Reverberation

1. **Direct Sound**: Sound travels directly from the source to the listener.
2. **Early Reflections**: Sound reflects off nearby surfaces and reaches the listener shortly after the direct sound.
3. **Late Reflections**: Multiple reflections build up and blend together, creating a complex, decaying sound field.



Reverberation

Obtaining Room Impulse Response (RIR) Samples

- Room Impulse Response (RIR) samples capture the acoustic characteristics of a space
- RIR samples are used in audio engineering and acoustics to simulate how sounds behave in specific environments

Sources for RIR Samples:

- **OpenAIR Library**: (http://www.openairlib.net/) offers a wide range of RIR samples from different acoustic environments.
- **Room Impulse Response and Noise Database SRL28** (https://www.openslr.org/28)

Other Applications of Reverberation than Data Augmentation for ASR

- Music Production
- Architectural Acoustics
- Virtual Reality and Gaming
- Audio Forensics

Noise Corruption

- adding noise to the audio signal
- mix the audio signal with noise samples
- noise samples can be obtained from various sources
  - **Musan**: (https://www.openslr.org/17/)
  - **Urban Sound Dataset**: (https://urbansounddataset.com/)
  - **Freesound**: (https://freesound.org/)
  - **AudioSet**: (https://research.google.com/audioset/)

Speed Perturbation

- **Speed Perturbation** involves altering the playback speed of an audio signal

Effects on the Audio Signal

1. **Speeding Up (Positive Perturbation)**:
    - The audio signal plays faster than the original.
    - **Perceived Pitch**: Increases; sounds higher.
    - **Duration**: Shortens; the total length of the audio decreases.
    - **Example**: Playing an audio file at 1.2x speed raises the pitch by app. 20%
2. **Slowing Down (Negative Perturbation)**:
    - The audio signal plays slower than the original.
    - **Perceived Pitch**: Decreases; sounds lower.
    - **Duration**: Lengthens; the total length of the audio increases.
    - **Example**: Playing an audio file at 0.8x speed lowers the pitch by app. 20%

Speed Perturbation

Detailed Effects on Perceived Pitch

- **Pitch and Speed Relationship**:
  - Pitch is directly proportional to speed
  - increasing the speed of an audio signal will result in a higher pitch
  - reducing the speed of an audio signal will result in a lower pitch
- **Mathematical Relationship**:
  - The frequency of a sound wave increases with speed perturbation: $f' = f \times k$
  - with f being the original frequency and k the speed factor

Music Example Calculation

- **Original Frequency**: 440 Hz (A4 note)
- **Speed Factor**: 1.5 (50% faster)
- **New Frequency**: $440 \times 1.5 = 660$ Hz (E5 note, which is higher)

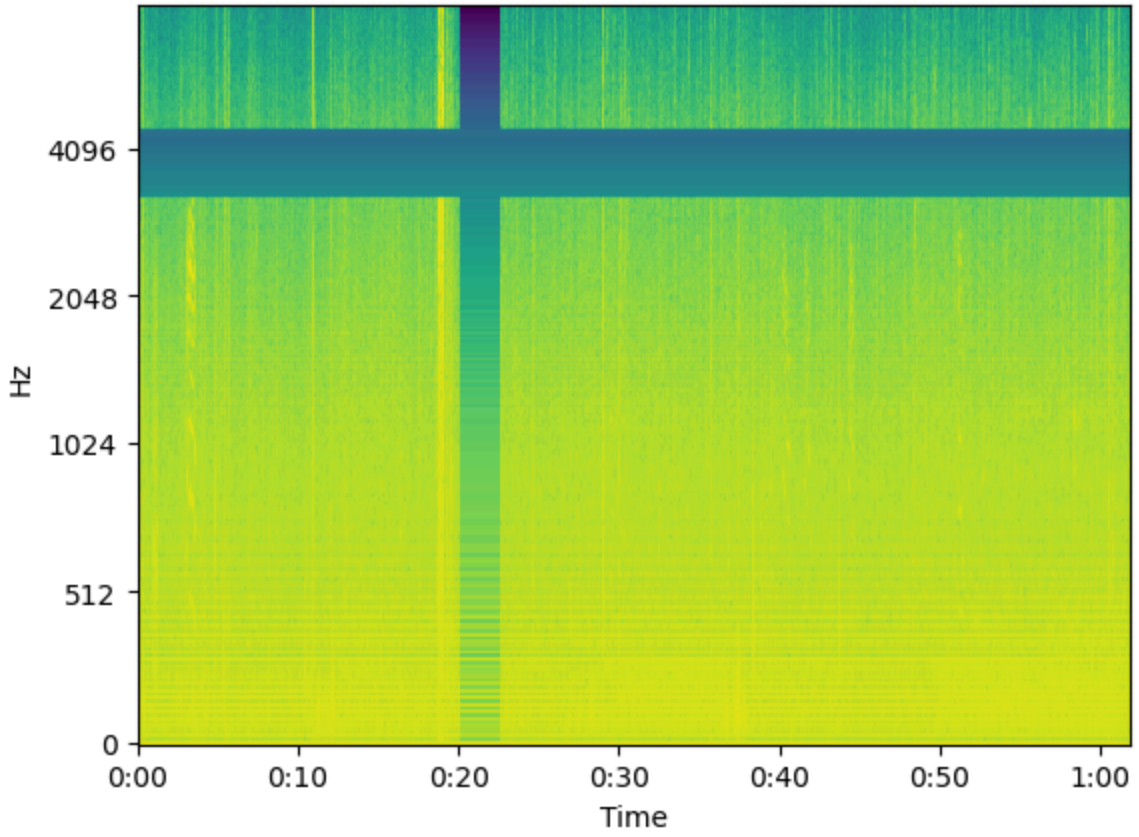
SpecAugment

Goals

- augmentation policy that acts on the log mel spectrogram directly
- help the network learn useful features
- features should be:
  - robust to deformations in the time direction
  - partial loss of frequency information
  - partial loss of small segments of speech

Three elements

- Time warping
- frequency masking
- time masking



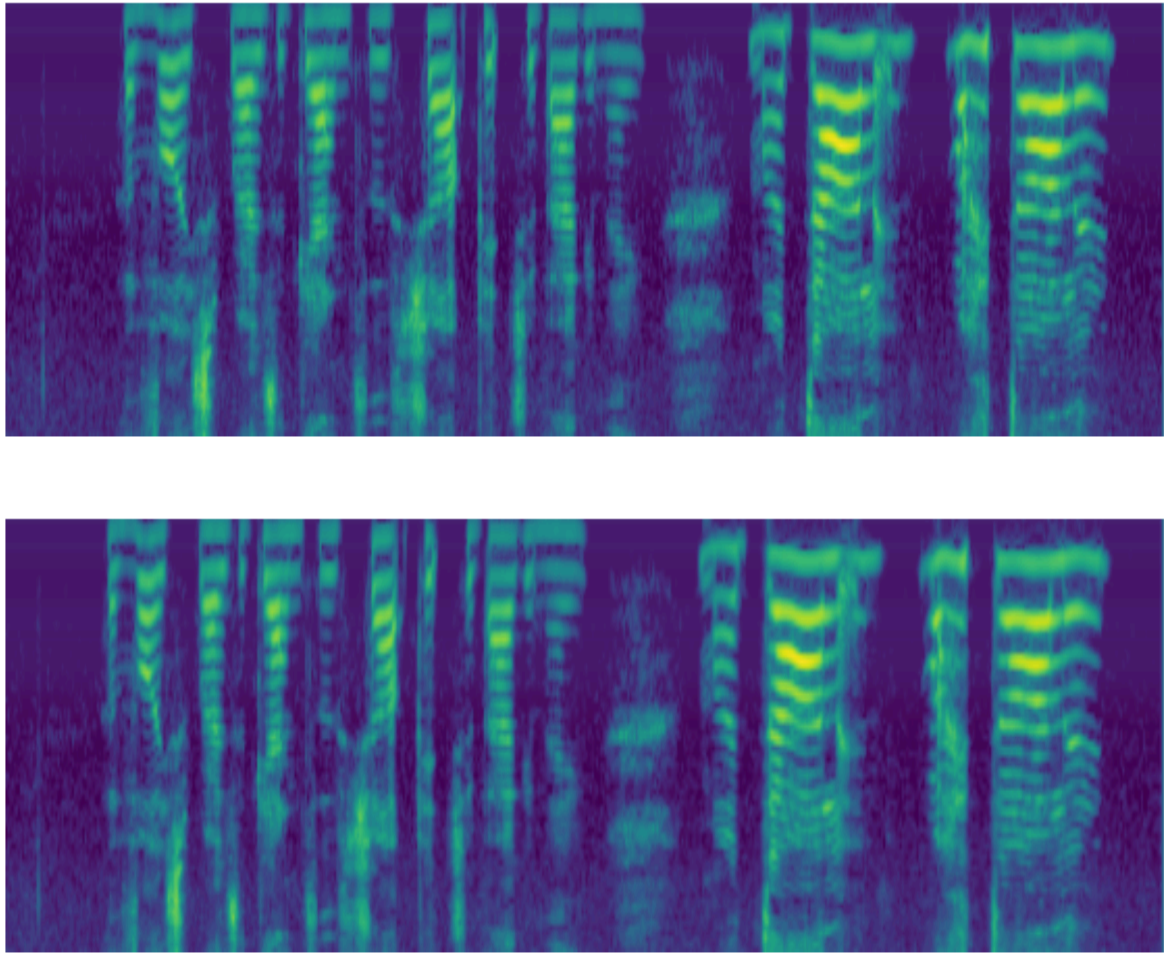
Park, D.S., Chan, W., Zhang, Y., Chiu, C.-C., Zoph, B., Cubuk, E.D., Le, Q.V., 2019. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition, in: Interspeech 2019. pp. 2613–2617. https://doi.org/10.21437/Interspeech.2019-2680

SpecAugment

Technische Hochschule Rosenheim

Time warping

- used the function sparse image warp of tensorflow
- given a log mel spectrogram with τ time steps
  - view the spetrogram as an image where the time axis is horizontal and the frequency axis is vertical
  - choosing a random point along the horizontal line passing through the center of the "image" (time steps $(W, \tau - W)$
  - warped either to the left or right by a distance w chosen from a uniform distribution from 0 to the time warp parameter W along that line.
  - fixing six anchor points on the boundary—the four corners and the mid-points of the vertical edges



Park, D.S., Chan, W., Zhang, Y., Chiu, C.-C., Zoph, B., Cubuk, E.D., Le, Q.V., 2019. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition, in: Interspeech 2019. pp. 2613–2617. https://doi.org/10.21437/Interspeech.2019-2680

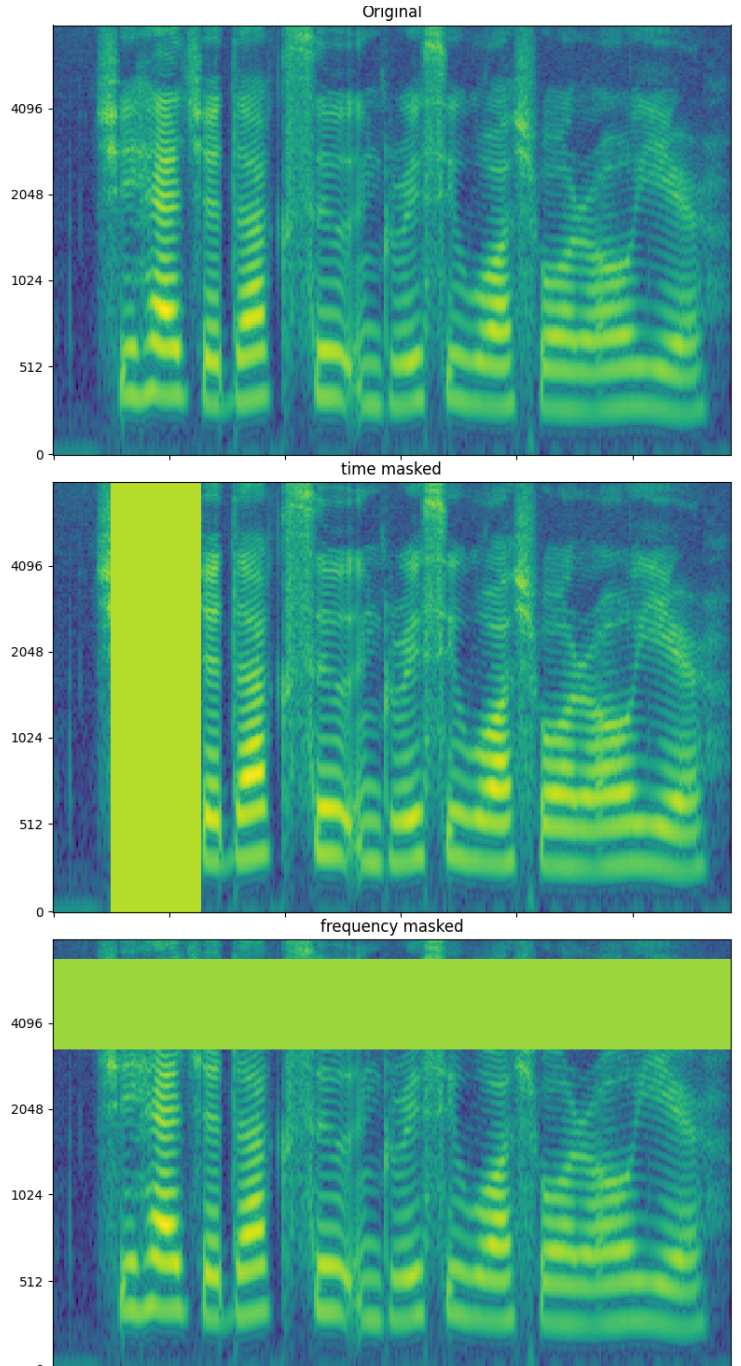
SpecAugment

Time Masking

- on t consecutive time steps $[t_0, t_0 + t)$
- t chosen from a uniform distribution from 0 to the time mask parameter T
- t_0 is chosen from $[0, \tau - t)$
- upper bound for masking

Frequency Masking

- f consecutive mel frequency channels $[f_0, f_0 + f)$ are masked
- f is chosen from a uniform distribution from 0 to the frequency mask parameter F
- f_0 is chosen from $[0,\ \nu - f)$
- ν is the number of mel frequency channels



Key Takeaways: Data Augmentation

- Deep models are data-hungry; augmentation **multiplies** effective training data and forces **robustness** to real-world variation
- **Waveform-level**: **reverberation** (convolve with room impulse responses), **noise corruption** (mix in MUSAN/AudioSet), **speed perturbation** (changes duration *and* pitch)
- **Feature-level**: **SpecAugment** masks the log-mel spectrogram directly — **time warping**, **frequency masking**, **time masking** — cheap and remarkably effective
- Mental model: don't just collect clean data, **simulate the messy conditions** the model will face

RNN-T: The Streaming Workhorse

Technische Hochschule Rosenheim

The third paradigm — what actually runs on your phone

- AED models (e.g. Whisper) are powerful but must see the **whole utterance** before decoding → useless for live captioning / voice assistants
- Plain CTC streams but **can't model output dependencies** (conditional independence assumption)
- **RNN-Transducer (Graves, 2012)** fixes both: keep CTC's frame-synchronous, streamable decoding **and** add an internal language model

⊙ Transducer (RNN-T)

A transducer couples three networks: an **encoder** (audio → acoustic features), a **prediction network** (previous tokens → a learned language-model state), and a **joint network** that fuses both to score the next token (incl. `blank`). It walks a $T \times U$ lattice, emitting tokens or advancing time — **without ever needing the full utterance.**

Why it won production

- **Streaming by design**: emits tokens as audio arrives, low latency
- **On-device friendly**: powers Google/Apple/Samsung dictation & assistants
- **Built-in LM**: the prediction network conditions on output history (unlike CTC)

Representative modern systems

- Google's streaming RNN-T (2019, first all-neural on-device ASR)
- **NVIDIA Parakeet / Canary** — current SOTA-class transducer models
- The encoder is usually a **Conformer** (next slide)

Streaming & Latency: The Production Constraint

- A lab system can wait for the whole utterance; a **voice assistant or live caption cannot** — it must emit words *as you speak*
- This single constraint is why **RNN-T won production** over the more accurate AED

Streaming vs offline

Offline: decode after the full utterance is available (max context, max accuracy). **Streaming**: emit tokens incrementally from a growing prefix of audio (low latency, slightly lower accuracy).

- **Why AED can't stream natively**: attention looks over *all* encoder frames before the first token — it needs the end of the utterance
- **CTC & RNN-T are frame-synchronous**: each new frame can immediately produce output

Real-Time Factor (RTF)

$$\text{RTF} = \frac{\text{processing time}}{\text{audio duration}}$$

RTF < 1 = faster than real time. Engineers tune **RTF *and* latency**, not just WER.

- A full-attention encoder also needs the whole signal $\rightarrow$ made streamable via **chunked / look-ahead attention**: attend only within a short window plus a small future context
- The trade-off is always the same: **more future context $\rightarrow$ better accuracy, worse latency**

Conformer: The De-Facto Modern ASR Encoder

- Encoders matter more than the loss: the **same encoder** can be trained with CTC, AED, *or* RNN-T
- **Transformers** capture **global** context via self-attention but are weak at fine **local** patterns
- **CNNs** capture local patterns (formant transitions, onsets) but have limited context
- **Conformer (Gulati et al., 2020)** = *convolution-augmented Transformer*: best of both, and it became the default ASR encoder

⊙ Conformer

A Conformer block sandwiches a **convolution module** (local features) between a **multi-head self-attention** module (global context) and two **macaron-style feed-forward** layers (half-step FFN before and after). Captures local + global acoustic structure in one block.

Why it matters for learning

- Self-attention → long-range dependencies (whole utterance)
- Conv module → local fine-grained acoustic detail
- This combination, not a new loss, drove much of the post-2020 WER drop
- Plug it into RNN-T → streaming SOTA; into AED → offline SOTA

Conceptual takeaway

Paradigm = how you align (CTC / AED / RNN-T).
Encoder = what extracts features (TDNN → RNN → Conformer).
The two choices are largely independent.

Gulati, A. et al., 2020. Conformer: Convolution-augmented Transformer for Speech Recognition, in: Interspeech 2020.

Key Takeaways: End-to-End Neural ASR

- **One network, audio $\rightarrow$ text** replaced the modular HMM-GMM pipeline; the central problem is aligning T frames to U tokens
- **Three paradigms**, and you should be able to contrast them:
  - **CTC** (Deep Speech, wav2vec 2.0 fine-tuning): per-frame labels + blank, conditionally independent, fast, *no internal LM*
  - **AED** (e.g. Whisper): encoder + attention + autoregressive decoder, strong implicit LM, *not streaming*
  - **RNN-T**: encoder + prediction net + joint net, *streaming + internal LM* $\rightarrow$ production default
- **Encoder vs. paradigm are independent choices**: encoders evolved TDNN $\rightarrow$ RNN $\rightarrow$ **Conformer** (local conv + global attention)
- **Output units**: modern systems decode **subwords** (BPE / SentencePiece), not characters — open-vocabulary and shorter targets U
- **Streaming vs. offline** is a first-class design axis: AED is offline-only, CTC/RNN-T are frame-synchronous $\rightarrow$ measure **latency / RTF**, not just WER
- **Data augmentation** (reverb, noise, speed, **SpecAugment**) is what makes these robust

Wav2Vec 2.0

- in this lecture, we already talked about self supervised learning...
- mostly for text data
- In **self supervised learning**, the objective is to first solve pre-text task with "pseudo-labels"
- easy in text: masked language model, next sentence prediction, next word prediction

⊙ Self-Supervised Learning (SSL)

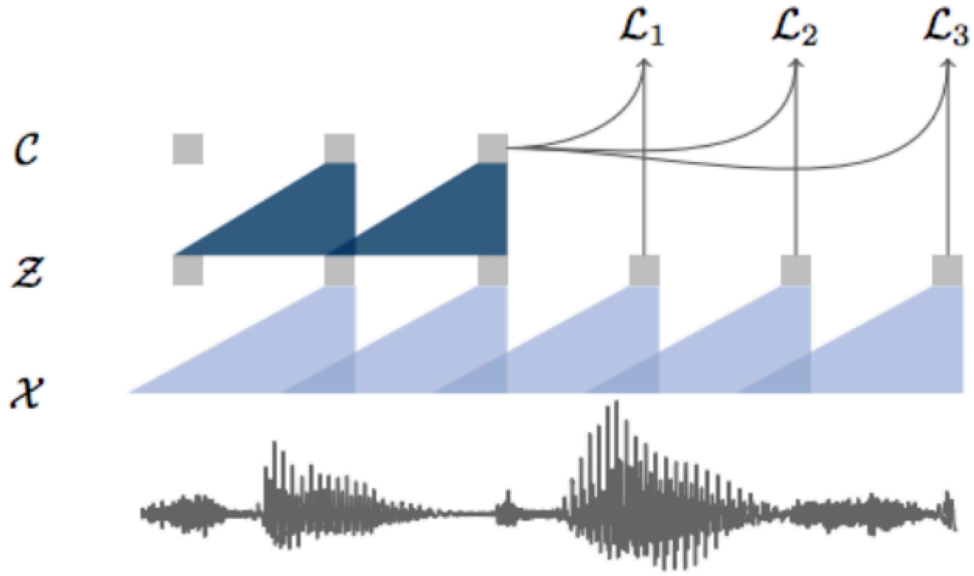
Pre-train a model on **huge amounts of unlabeled** audio by solving a made-up "pre-text" task (predict masked/future content) → learn general speech representations. Then **fine-tune** on a *small* labeled set for ASR. Decouples representation learning from expensive transcription. wav2vec 2.0, HuBERT, and WavLM are all instances of this one paradigm.

Wav2Vec 2.0

Gradual Development of self-supervision for audio / ASR

wav2vec: Latent speech audio representations

- Fully convolutional architecture
- raw audio input
  - encoder network (~30ms auf audio)
  - context network (~210ms of audio)
- Trained with contrastive loss, predicting future samples
- Minimizing the contrastive loss by distinguish samples from the future from distractors sampled from a distribution of negative samples
- Representations used to improve ASR



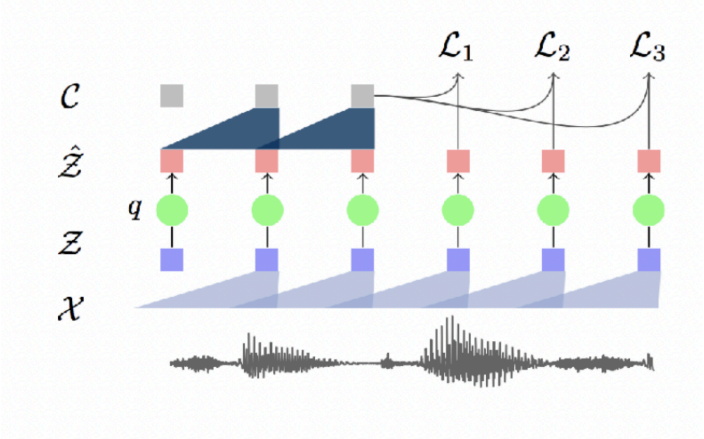
Source: Schneider, S., Baevski, A., Collobert, R., Auli, M., 2019. wav2vec: Unsupervised Pre-training for Speech Recognition.

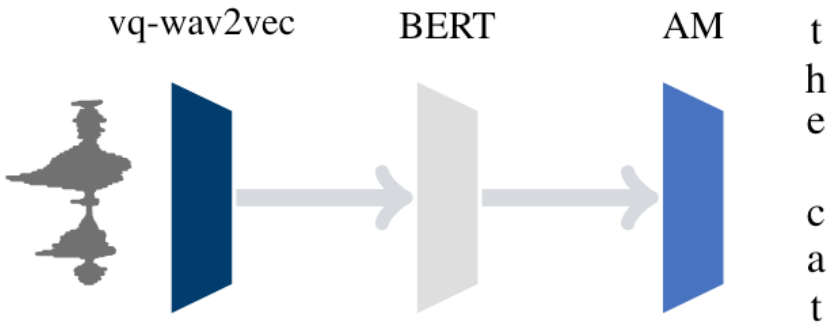
Wav2Vec 2.0

Gradual Development of self-supervision for audio / ASR

vq-wav2vec: Self-Supervised Learning of Discrete Speech Representations

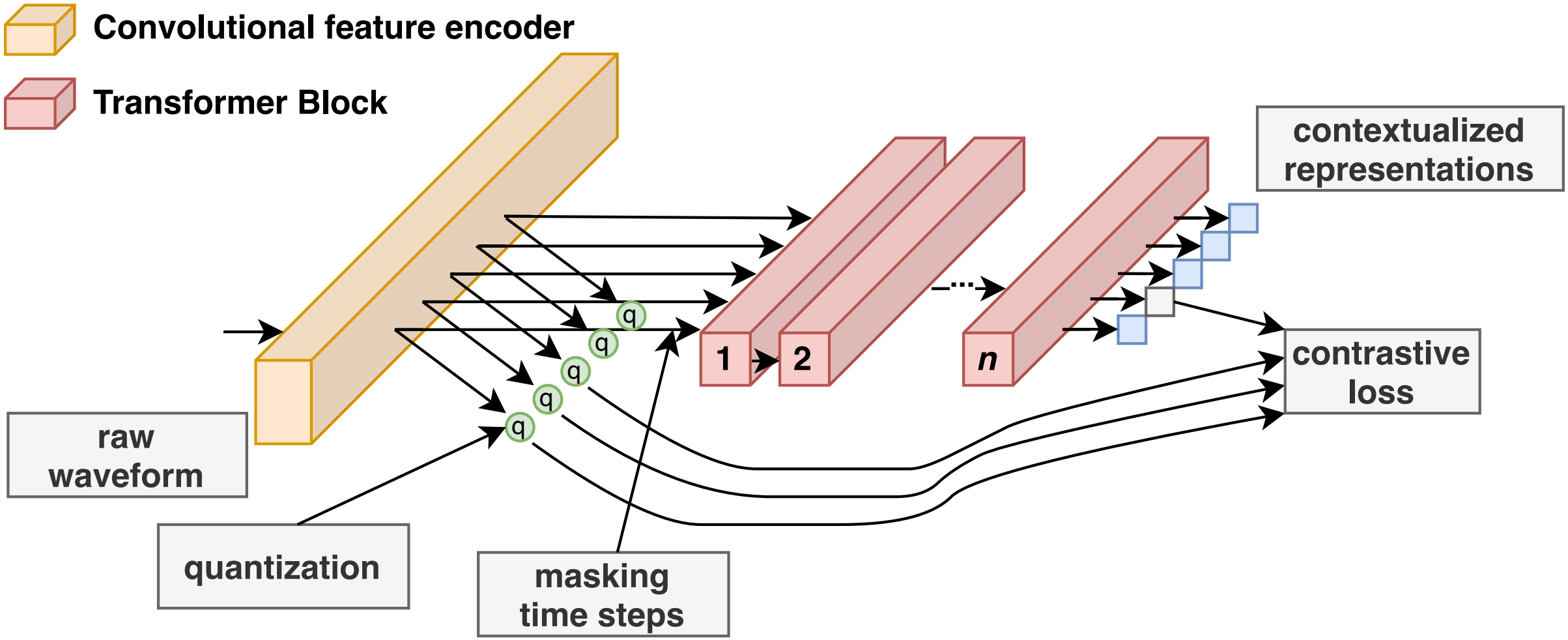
- vq-wav2vec encoder maps raw audio X to a dense representation Z
- quantized (q) to $\hat{Z}$
- aggregated into context representations (C)
- training requires future time step prediction
- acoustic models are trained with the quantized representations, then applying BERT (transformer) to the discretized sequence
- feeding the resulting representations into the acoustic model to output transcriptions.





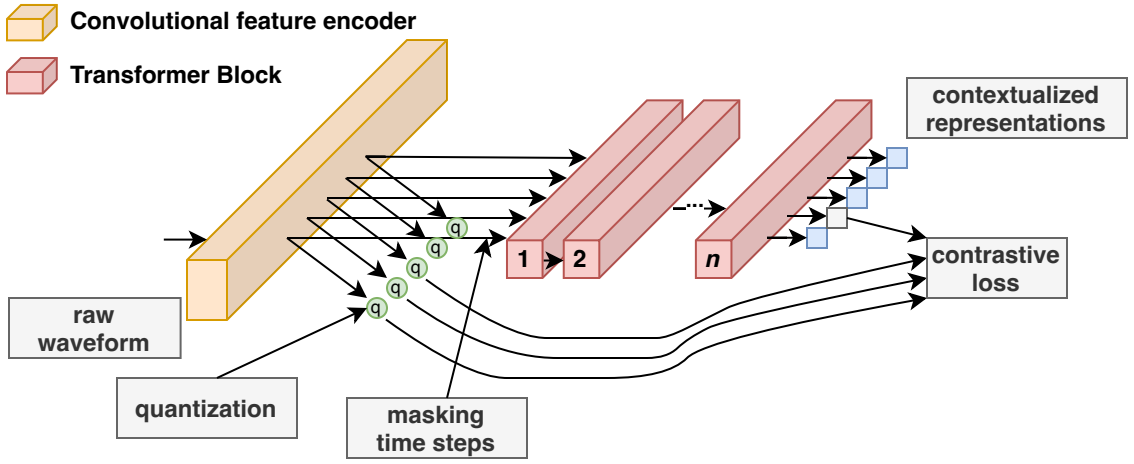
Source:Baevski, A., Schneider, S., Auli, M., 2020. vq-wav2vec: Self-Supervised Learning of Discrete Speech Representations.

Wav2Vec 2.0


Convolutional feature encoder
Transformer Block
contextualized representations
raw waveform
quantization
masking time steps
contrastive loss
q
q
q
q
q
1
2
n

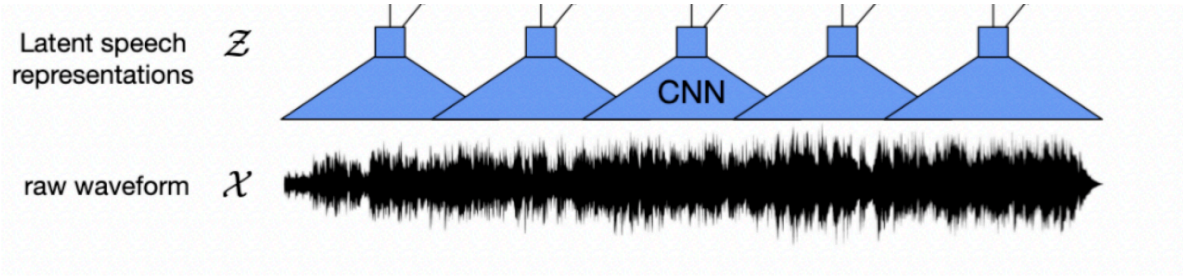
Wav2Vec 2.0

- Joint VQ & context representation learning
- Bi-directional-contextualized representations
- Contrastive task
- Vector quantized targets
- Fine-tuned on labeled data



Wav2Vec 2.0

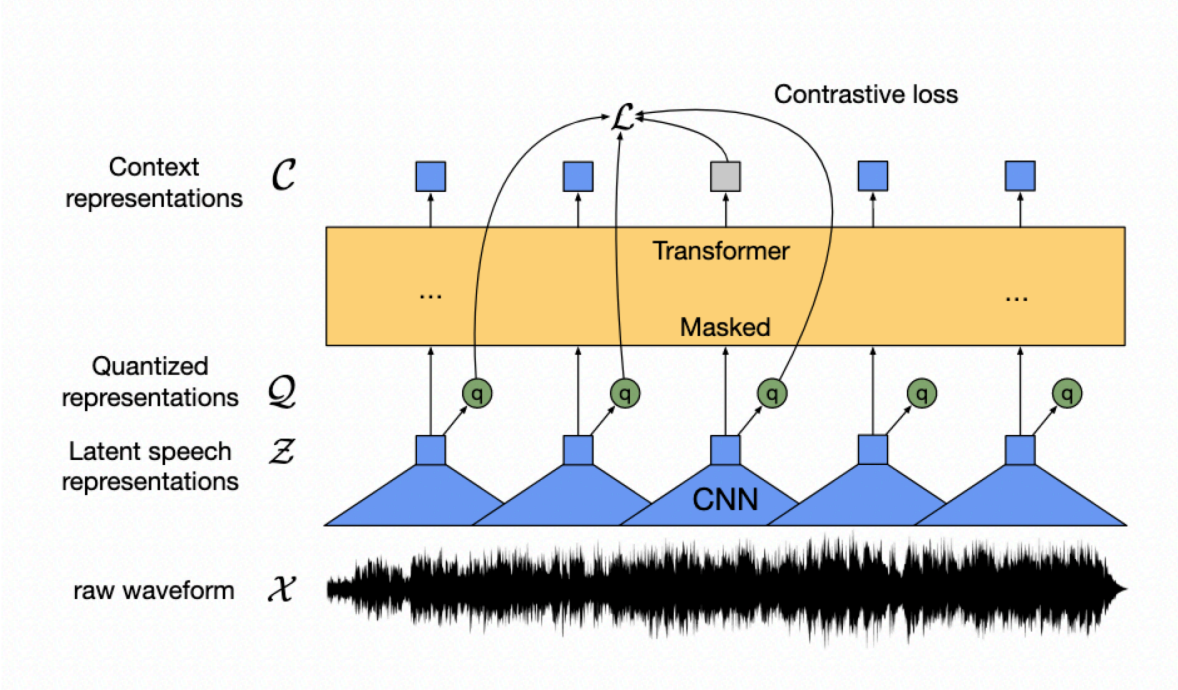
- input raw waveform
- multi-layer convolutional feature encoder



Baevski, A., Zhou, Y., Mohamed, A., Auli, M., 2020. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations, in: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M.F., Lin, H. (Eds.), Advances in Neural Information Processing Systems. Curran Associates, Inc., pp. 12449–12460.

Wav2Vec 2.0

- input raw waveform
- multi-layer convolutional feature encoder
- quantization
- masking of quantized sequence (similar to Masked language modeling)
- Transformer Network to build contextualized representations
- contrastive loss $\rightarrow$ true latent vs distractors



Baevski, A., Zhou, Y., Mohamed, A., Auli, M., 2020. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations, in: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M.F., Lin, H. (Eds.), Advances in Neural Information Processing Systems. Curran Associates, Inc., pp. 12449–12460.

Pre-Training Loss Function Wav2Vec 2.0: Contrastive Loss + Diversity Loss

1. Contrastive Loss (Main Objective)

Then the **contrastive loss** for a single time step is:

$$\mathcal{L}_{\text{contrastive}} = -\log \frac{\exp(\text{sim}(c_t, q_t)/\kappa)}{\sum_{q' \in Q \cup \{q_t\}} \exp(\text{sim}(c_t, q')/\kappa)}$$

- z_t is the encoder output at time t (before masking)
- c_t is the **contextualized representation** after the transformer (with masking)
- q_t is the **quantized target** (true code) at the masked time step
- Q is a set of negative samples (wrong codes).

With:

- $\text{sim}(a, b)$ is the cosine similarity between vectors a and b
- κ is a temperature parameter (to sharpen the distribution)

Goal: Maximize similarity between c_t and true q_t, minimize similarity with negatives.

Pre-Training Loss Function Wav2Vec 2.0: Contrastive Loss + Diversity Loss

2. Codebook Diversity Loss (Optional Regularization)

- used to ensure that the model uses **all entries** in the quantization codebook
- avoid collapse to a few codes

$\mathcal{L}_{\text{diversity}}$, which uniform usage of the quantization slots

$$\mathcal{L}_{\text{diversity}} = \frac{1}{GV} \sum_{g=1}^{G} (V - \|p_g\|_0)$$

Where:

- G = number of codebooks (groups)
- V = number of entries per codebook
- p_g is the full categorical distribution over **all** V **entries** of codebook g

Pre-Training Loss Function Wav2Vec 2.0: Contrastive Loss + Diversity Loss

3. Final Loss

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{contrastive}} + \alpha \cdot \mathcal{L}_{\text{diversity}}$$

Where:

- α is a weighting factor (typically set to 0.1 or 0.2)

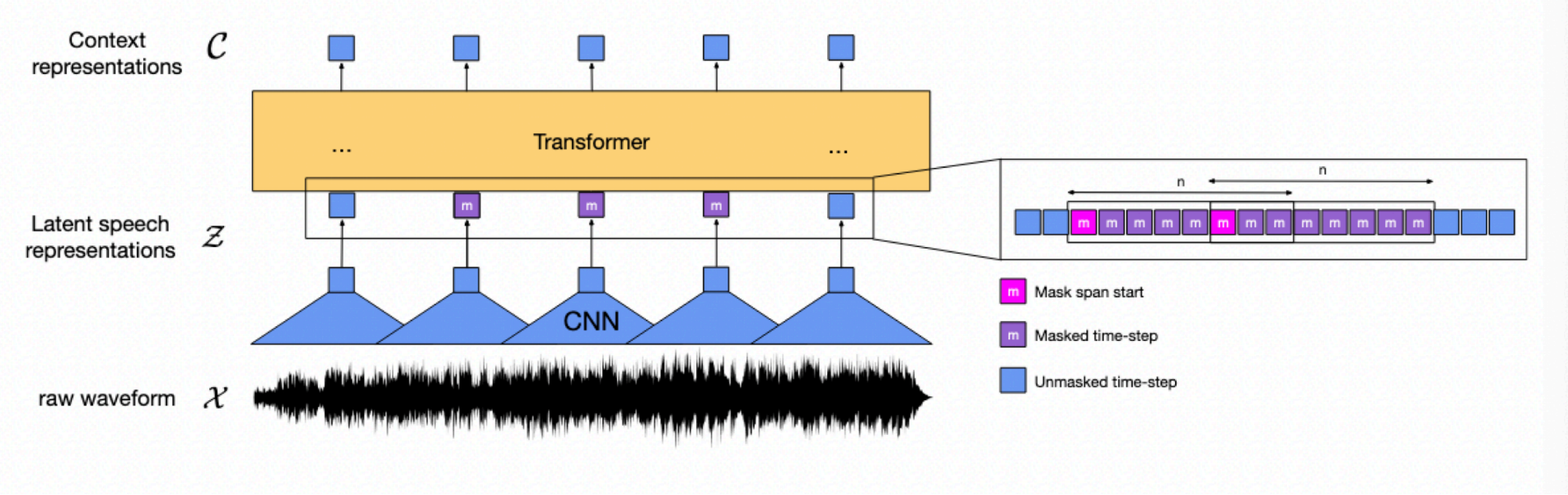
Summary of Wav2Vec 2.0 Loss Components

Component	Purpose
Contrastive Loss	Forces model to predict correct masked quantized tokens.
Diversity Loss	Ensures usage of all quantization codes.
Masked Prediction	Like BERT—model fills in masked audio segments.

Wav2Vec 2.0

Masking

- Sample starting points for masks without replacement,
- then expand to 10 time steps
- sampled spans can overlap
- e.g., for a 15s sample, $\approx 49\%$ of time steps are
- average span length of ~300ms (compare: BERT: $\approx 15\%$ of tokens)



Wav2Vec 2.0

Model types:

- Identical convolutional feature encoder:
  - seven blocks with temporal convolutions in each block that have 512 channels
  - with strides (5,2,2,2,2,2,2) and kernel widths (10,3,3,3,3,2,2).
  - encoder output frequency is 49Hz, i.e., context of 25ms and 20ms stride

Transformer Configuration

- **Base**: 12 layers, 768 dim
- **Large**: 24 layers, 1024 dim

Xross-lingual and monolingual models

Wav2Vec 2.0

Training Data

Unlabled Data

- LibriSpeech 960h (LS-960), LibriVox(LV-60k) net, 53.2k audio

Labeled Data

- Different settings from the unlabeled data
- TIMIT for phone recognition

More info on models:

https://github.com/facebookresearch/fairseq/tree/main/examples/wav2vec#wav2vec-20

Wav2Vec 2.0

Fine tuning for ASR:

- Add a single linear projection on top
- map into target vocab and train with CTC loss with a low learning rate
- CNN encoder is usually not trained, i.e. frozen
- Use modified **SpecAugment** in latent space to prevent early overfitting
- Uses **wav2letter** decoder with the official 4-gram LM and Transformer LM
- any other decoder can be used, this is where you can customize easily

Wav2Vec 2.0

Results

Table 10: WER on Librispeech when using all 960 hours of Librispeech as labeled data (cf. Table 2).

Model	Unlabeled data	LM	dev		test	
			clean	other	clean	other
LARGE - from scratch	-	None	2.8	7.6	3.0	7.7
	-	4-gram	1.8	5.4	2.6	5.8
	-	Transf.	1.7	4.3	2.1	4.6
BASE	LS-960	None	3.2	8.9	3.4	8.5
		4-gram	2.0	5.9	2.6	6.1
		Transf.	1.8	4.7	2.1	4.8
LARGE	LS-960	None	2.6	6.5	2.8	6.3
		4-gram	1.7	4.6	2.3	5.0
		Transf.	1.7	3.9	2.0	4.1
LARGE	LV-60k	None	2.1	4.5	2.2	4.5
		4-gram	1.4	3.5	2.0	3.6
		Transf.	1.6	3.0	1.8	3.3

Wav2Vec 2.0

Results

Table 2: WER on Librispeech when using all 960 hours of labeled data (cf. Table 1).

Model	Unlabeled data	LM	dev		test	
			clean	other	clean	other
Supervised						
CTC Transf [51]	-	CLM+Transf.	2.20	4.94	2.47	5.45
S2S Transf. [51]	-	CLM+Transf.	2.10	4.79	2.33	5.17
Transf. Transducer [60]	-	Transf.	-	-	2.0	4.6
ContextNet [17]	-	LSTM	1.9	3.9	1.9	4.1
Conformer [15]	-	LSTM	2.1	4.3	1.9	3.9
Semi-supervised						
CTC Transf. + PL [51]	LV-60k	CLM+Transf.	2.10	4.79	2.33	4.54
S2S Transf. + PL [51]	LV-60k	CLM+Transf.	2.00	3.65	2.09	4.11
Iter. pseudo-labeling [58]	LV-60k	4-gram+Transf.	1.85	3.26	2.10	4.01
Noisy student [42]	LV-60k	LSTM	1.6	3.4	1.7	3.4
This work						
LARGE - from scratch	-	Transf.	1.7	4.3	2.1	4.6
BASE	LS-960	Transf.	1.8	4.7	2.1	4.8
LARGE	LS-960	Transf.	1.7	3.9	2.0	4.1
	LV-60k	Transf.	1.6	3.0	1.8	3.3

Wav2Vec 2.0

Results

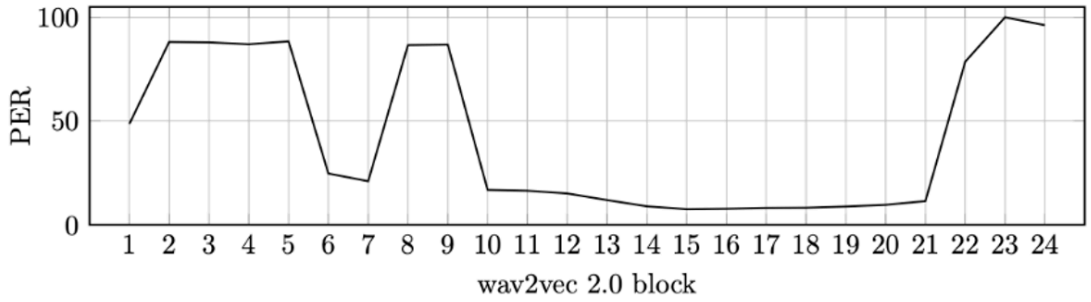


Figure 2: Supervised phoneme classification using representations from different wav2vec 2.0 blocks on dev-other of English Librispeech. Low and high blocks do not provide good features, while as blocks 14-19 do. Block 15 performs best.

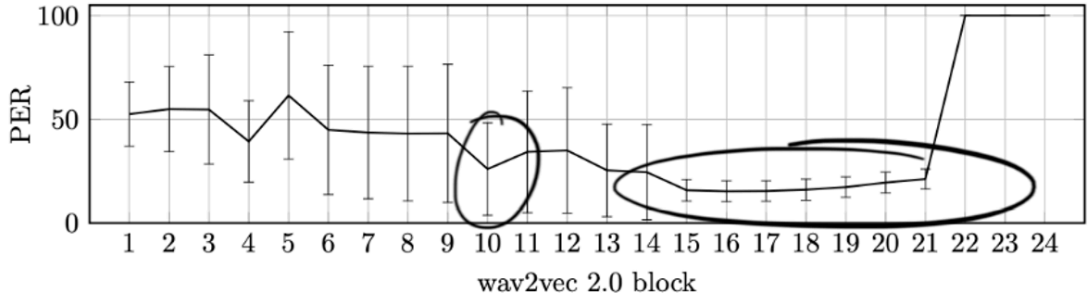


Figure 3: Supervised phoneme classification on eight languages of the MLS dataset in terms of mean PER and standard deviation for different wav2vec 2.0 blocks to represent the raw audio (cf. Figure 2). We consider English, German, Spanish, French, Italian, Dutch, Polish and Portuguese.

Wav2Vec 2.0

- used for many other use cases other than ASR
- **speaker recognition**: Fan, Z., Li, M., Zhou, S., Xu, B., 2021. Exploring wav2vec 2.0 on speaker verification and language identification.
- **speech emotion recognition**: Pepino, L., Riera, P., Ferrer, L., 2021. Emotion Recognition from Speech Using wav2vec 2.0 Embeddings
- **stuttering detection**: Bayerl, S.P., Wagner, D., Noeth, E., Riedhammer, K., 2022. Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0
- **Alzheimer's classification**: Braun, F., Erzigkeit, A., Lehfeld, H., Hillemacher, T., Riedhammer, K., Bayerl, S.P., 2022. Going Beyond the Cookie Theft Picture Test: Detecting Cognitive Impairments using Acoustic Features
- ...

The SSL Family: One Paradigm, Three Recipes

wav2vec 2.0 is not alone — same idea, different pre-text task

wav2vec 2.0 (2020)

- **Contrastive** task: pick the true quantized latent vs. distractors
- Learns to tell the real masked code from imposters

HuBERT (2021)

- **Masked prediction** of *cluster IDs*: k-means on features gives pseudo-labels, predict them like a masked-LM (BERT for audio)
- Iterative: re-cluster better features, repeat

WavLM (2022)

- HuBERT-style masked prediction **+ denoising**: mixes in overlapping speech/noise during pre-training
- Adds **speaker-aware** objectives → strong not just for ASR but diarization, speaker ID, separation

Key insight

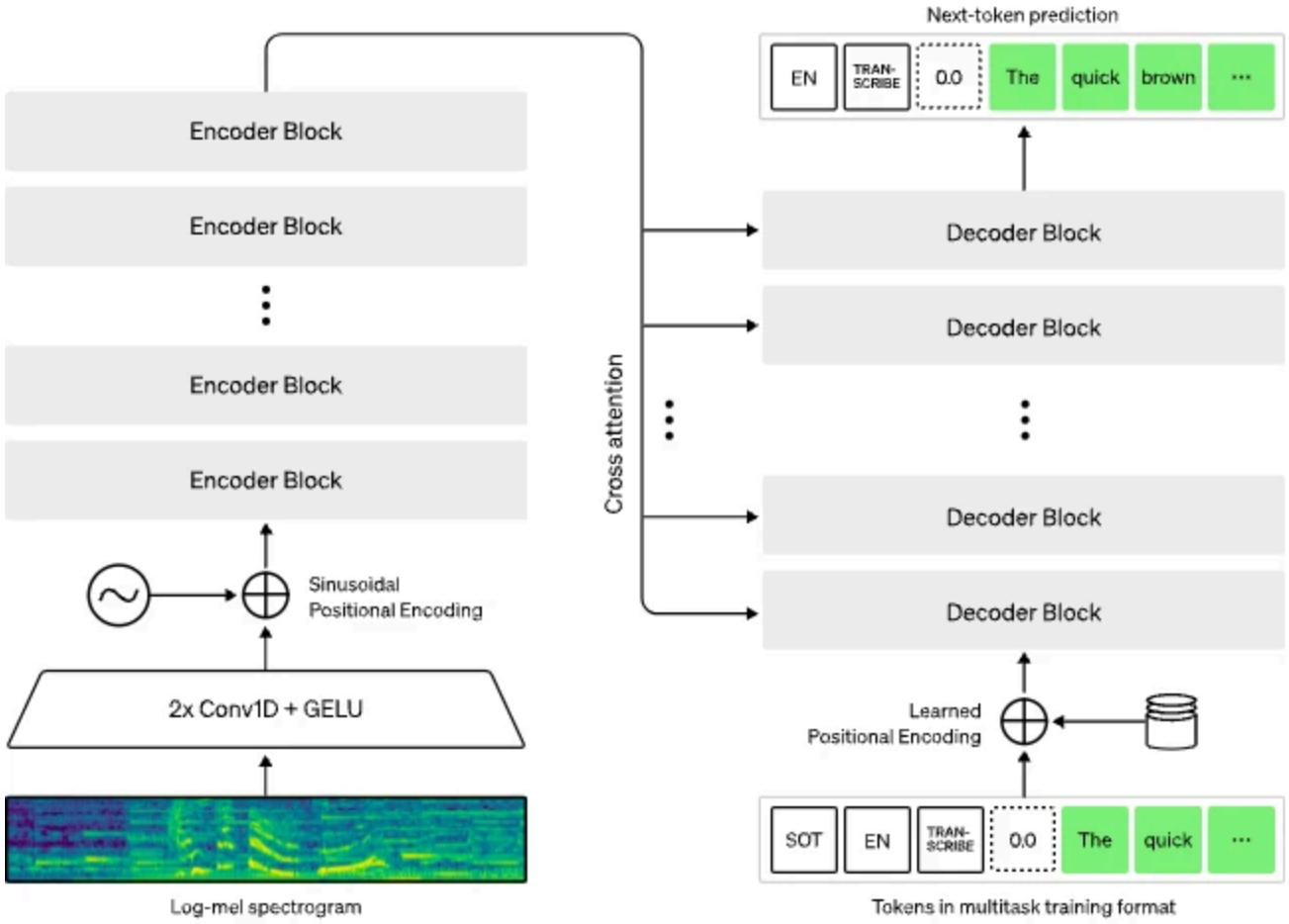
The *encoder* is general-purpose. Swap the **pre-text task** to bias what it learns: contrastive (w2v2), cluster-prediction (HuBERT), or denoising + speaker info (WavLM). All fine-tune to ASR with a thin CTC/RNN-T head.

Hsu et al., 2021. HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. · Chen et al., 2022. WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing.

Key Takeaways: Self-Supervised Learning

- **The core idea**: pre-train on **mountains of unlabeled audio** (a pre-text task), then fine-tune on a *little* labeled data — breaks the dependency on expensive transcriptions
- **wav2vec 2.0**: CNN encoder + quantization + Transformer, trained with a **contrastive** loss on masked frames; fine-tune with a thin **CTC** head
- The **same paradigm**, different pre-text task: **HuBERT** (predict cluster IDs, BERT-style) and **WavLM** (masked prediction + denoising + speaker awareness)
- Learned representations transfer **far beyond ASR**: speaker ID, emotion, stuttering/health detection, diarization
- **Why it mattered**: state-of-the-art ASR with a *fraction* of the labeled data

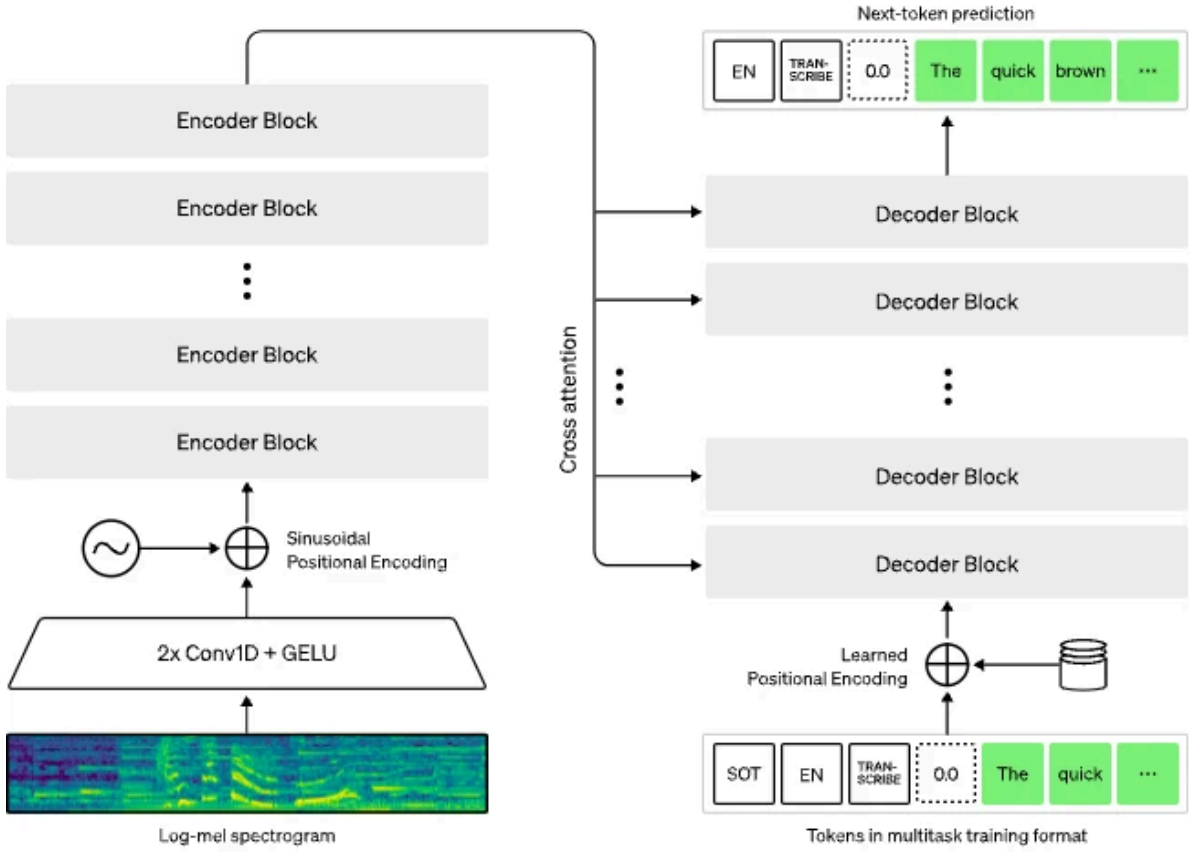
Whisper



Source: https://openai.com/index/whisper/

Whisper

- trained on 680,000 hours of multilingual and multitask supervised data collected from the web
- improved robustness to accents, background noise and technical language
- Whisper architecture is a simple end-to-end approach, implemented as an encoder-decoder Transformer
- audio split into 30-second chunks, converted into a log-Mel spectrogram, and then passed into an encoder
- decoder is trained to predict the corresponding text caption intermixed with special tokens
- the model also performs:
  - language identification
  - phrase-level timestamping
  - multilingual speech transcription
  - and to-English speech translation

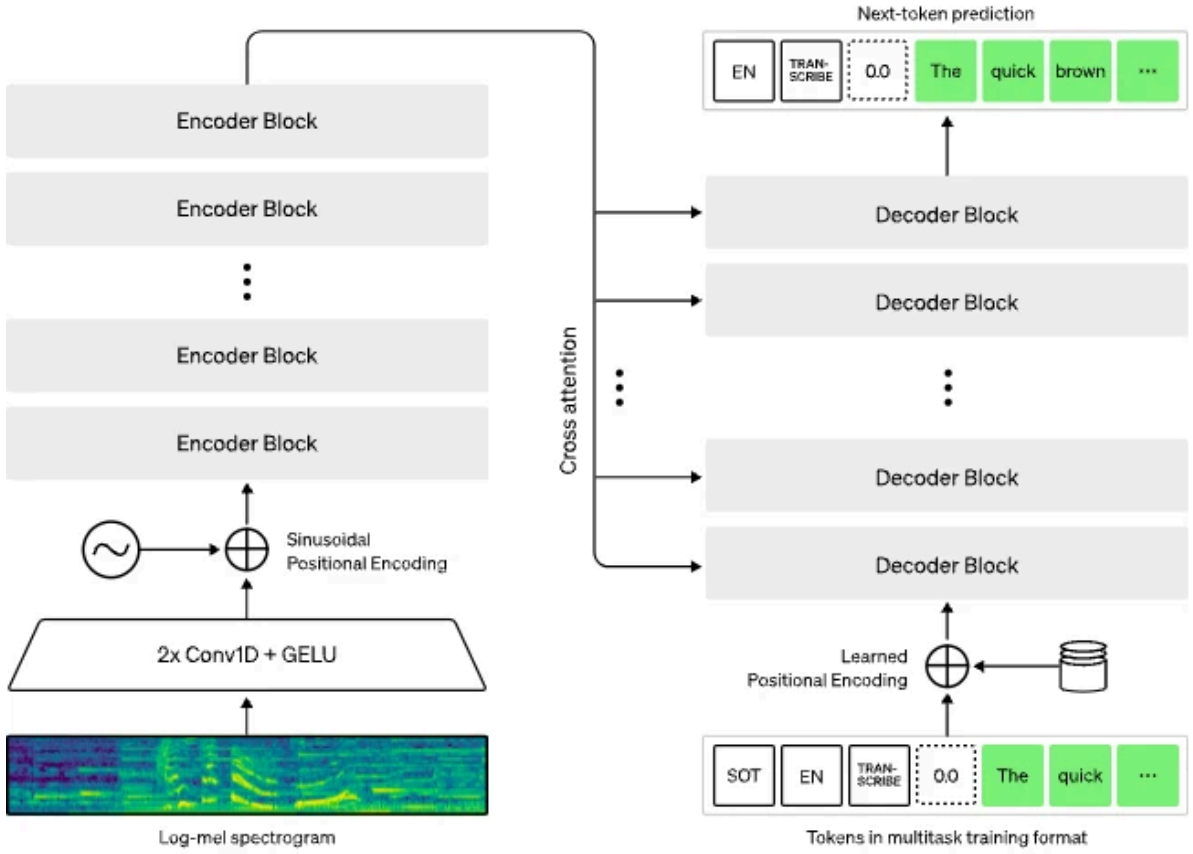


Source: https://openai.com/index/whisper/

Whisper

Model

- focus is on studying large-scale supervised pre-training for speech recognition
- architecture is "off-the-shelf" Transformer encoder-decoder
- convolutional feature-extraction before the transformer encoder
- GPT byte-level BPE tokenizer
- **Input data**:
  - clever 'filtering' and selection of web data, heuristics based
  - 16kHz audio
  - 80-channel log-magnitude Mel spectrogram, 25ms frame size, 10ms frame shift
  - feature normalization, scaling -1 and 1, approx 0-mean across the dataset



Whisper

Model Configurations

Model	Layers	Width	Heads	Parameters
Tiny	4	384	6	39M
Base	6	512	8	74M
Small	12	768	12	244M
Medium	24	1024	16	769M
Large	32	1280	20	1550M

Whisper

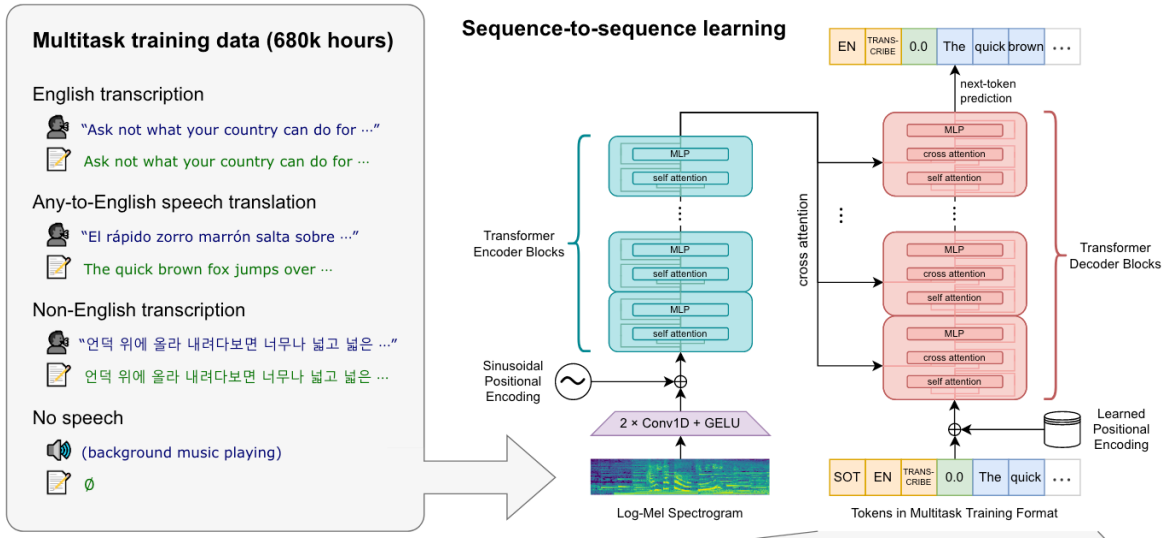
Multitask Format

- full ASR systems often have independent components for:
- voice activity detection (VAD)
- speaker diarization
- inverse text normalization (ITN)

→ **Model should perform the whole pipeline**

additional tasks:

- translation
- alignment
- language identification

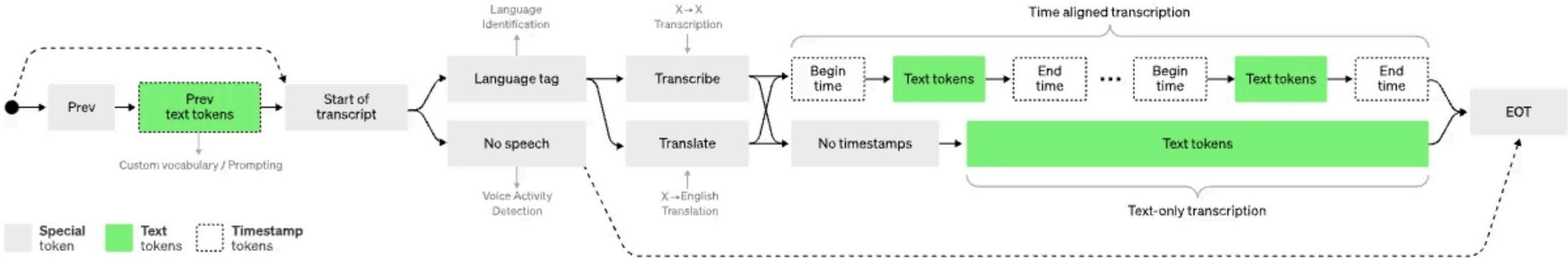


Whisper

Multitask Format

- simple format to specify all tasks and conditioning information as a sequence of input tokens (to the decoder)
- the decoder is an audio-conditional language model
- decoder is trained to condition on the history of text of the transcript
  - should learn longer-range text context to resolve ambiguous audio
  - with some probability the transcript text preceding the current audio segment is added to the decoders context
  - indicating the beginning of prediction with a `<startoftranscript>` token.
    - first predict the language (represented by a unique token)
    - if no speech: ` token
    - next token specifies the task `<transcribe>` or `<translate>` token
    - predict timestamps or not `<notimestamps>` token
- interleave prediction with the caption tokens
- Lastly, we add a token. We only mask out the training loss over the previous context text, and train the model to predict all other tokens. Please see Figure 1 for an overview of our format and training setup.

Whisper





Source: https://openai.com/index/whisper/

Whisper

Results

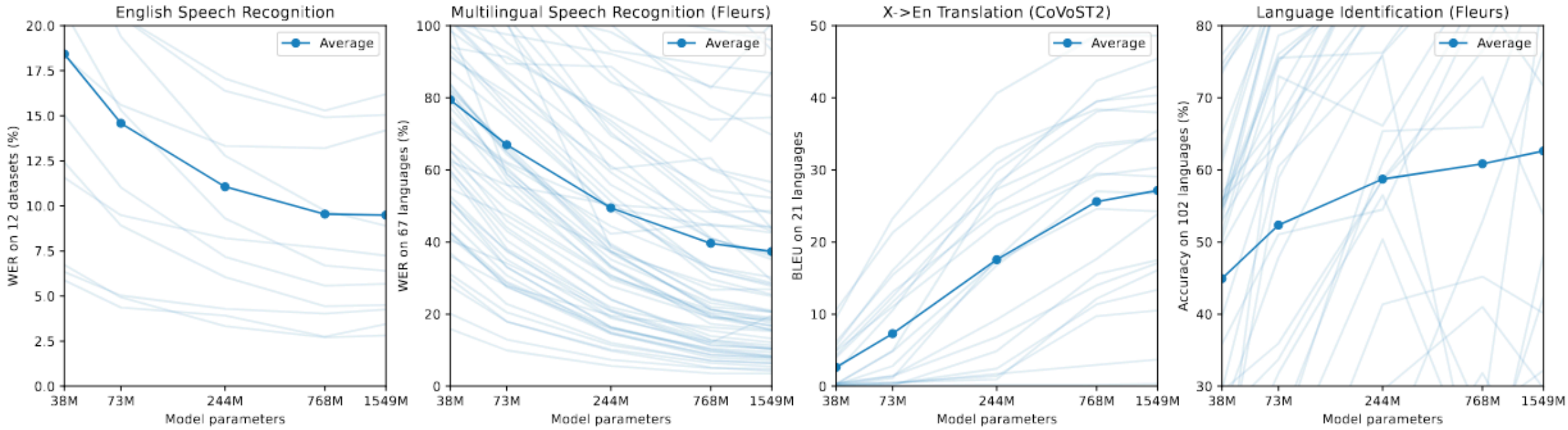


Figure 8. **Zero-shot Whisper performance scales reliably across tasks and languages with increasing model size.** Lightly shaded lines represent individual datasets or languages, showing that performance is more varied than the smooth trends in aggregate performance.

→ **more results in: Radford et al., 2022. Robust Speech Recognition via Large-Scale Weak Supervision**

Whisper

Results

	TED-LIUM3	Meanwhile	Kincaid46	Rev16	Earnings-21	Earnings-22	CORAAL	Average
Greedy decoding only	**3.55**	5.95	9.96	12.1	10.5	13.9	21.3	11.0
+ Beam search	3.66	5.78	9.18	12.1	10.2	13.5	20.1	10.6
+ Temperature fallback	3.66	5.72	8.93	12.0	10.2	13.4	**19.8**	10.5
+ Previous-text conditioning	3.68	5.35	9.19	11.3	9.78	**12.8**	**19.8**	10.3
+ Voice activity detection	3.57	**5.28**	9.16	10.8	9.75	12.9	19.9	10.2
+ Initial timestamp constraint	3.56	5.33	**8.87**	**10.6**	**9.68**	13.0	19.9	**10.1**

Table 7. Long-form transcription performance improves incrementally as additional decoding heuristics are employed. Details on each intervention are described in Section 4.5.

Speech meets LLMs

Where ASR is heading (2023 - today)

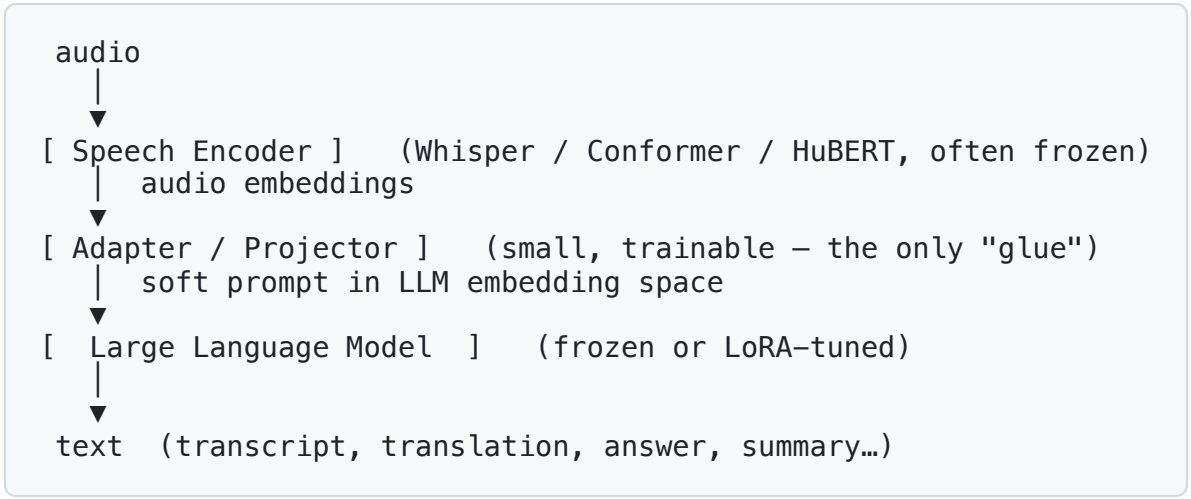
- Whisper's lesson: **scale + weak supervision** beats clever pipelines, and a Transformer decoder is *already* a language model
- Logical next step: **don't train a small ASR decoder — bolt the audio onto a full pretrained LLM**
- Idea: treat audio as **just another "language"** the LLM can read, so all the LLM's reasoning/knowledge/instruction-following comes along for free

⊙ Speech-LLM (audio-conditioned LLM)

A pretrained **LLM** (e.g. Llama/Qwen) is connected to a **speech encoder** (often a frozen Whisper/Conformer/HuBERT) through a small trainable **projector/adapter** that maps audio embeddings into the LLM's token-embedding space. The LLM then "reads" speech and responds in text — enabling not just transcription but **instruction-following over audio.**

Speech meets LLMs

The architecture pattern



- The adapter is tiny; most capability is **inherited** from the two pretrained giants
- Train on (audio, instruction, text) triples

What this unlocks

- **Instruction-following ASR**: "transcribe and fix grammar", "translate to German"
- **Spoken QA / summarization** directly from audio
- **In-context biasing**: paste a glossary of names into the prompt
- One model, many tasks (echoing Whisper's multitask idea, now general-purpose)

Representative systems

- **Qwen-Audio / Qwen2-Audio**, **SALMONN** (open)
- **GPT-4o**, **Gemini** native-audio (proprietary, end-to-end speech-in)

Speech meets LLMs - Challenges

Open challenges

Streaming & latency (LLMs are big), **hallucination** (LLMs may "transcribe" plausible-but-wrong words), and **evaluation** beyond WER (instruction-following has no single ground truth).

Key Takeaways: Foundation Models & the Frontier

- **Whisper**: a plain Transformer encoder-decoder (**AED**) trained on **680k h** of weakly-supervised web data → huge **robustness** and **multitask** ability (transcribe, translate, language ID, timestamps) from one model
- The lesson: **scale + weak supervision** beats clever hand-engineering — and the decoder is already a language model
- **Speech meets LLMs** (the current frontier): connect a **speech encoder** to a pretrained **LLM** via a small **adapter** → instruction-following over audio, not just transcription
- Representative systems to know: **Qwen-Audio**, **SALMONN**, **GPT-4o / Gemini** native audio
- Open problems remain: **streaming latency**, **hallucination**, and **evaluation beyond WER**

Which transcription is better?

Spontaneous speech, two systems

0:00 / 0:00

Reference (human transcript)

> [PLACEHOLDER: reference transcript]

- Which one would you rather read?
- Which one is better for **what** you want to do with it?

System A

> [PLACEHOLDER: hypothesis A]

System B

> [PLACEHOLDER: hypothesis B]

Same WER, different meaning?

	System A	System B
WER	[PLACEHOLDER]	[PLACEHOLDER]
CER	[PLACEHOLDER]	[PLACEHOLDER]
BERTScore (F1)	[PLACEHOLDER]	[PLACEHOLDER]
Audience vote	[PLACEHOLDER]	[PLACEHOLDER]

- WER counts every substitution, deletion and insertion **equally**
- One meaning-changing word costs the same as a harmless one:
  - fillers and disfluencies ("uh", "you know")
  - inflection or spelling variants ("gonna" vs. "going to")
  - a dropped negation ("is" vs. "isn't")
- The metric ranking and the human ranking can disagree

Beyond WER, PER and CER

Lexical (surface) metrics

- **WER / PER / CER**: edit distance on words, phones or characters
- Cheap, reproducible, comparable across papers
- Blind to meaning: paraphrases are errors, small meaning flips are not

Semantic and concept-level metrics

- **BERTScore**: token similarity from contextual embeddings (e.g. BERT) between reference and hypothesis, so paraphrases and semantic closeness count
- **Concept Error Rate / intent accuracy**: only the task-relevant parts must be right, e.g. named entities, key action verbs, medical conditions, dialogue intents

⊙ Evidence: do metrics agree with humans?

On side-by-side human preference judgements (HATS-en, built on LibriSpeech test-clean), the metric's choice agreed with the human consensus in **75.5 %** of the cases for WER and **86.3 %** for CER. Semantic metrics (BERTScore, SemDist) also beat WER, but the best one was not significantly better than CER.
The authors recommend reporting CER by default plus a validated semantic metric.

- **Choose the metric for the task**: dictation, search, medical notes and voice commands care about different errors
- **Always read some errors**: a single number hides what went wrong

Sharma et al., 2026. Rethinking Human-Aligned Evaluation: An Analysis of Semantic Metrics Beyond WER. arXiv:2609.21663. The paper compares WER, CER, BERTScore and SemDist; concept error rate / intent accuracy are not part of it.

Technische Hochschule Rosenheim

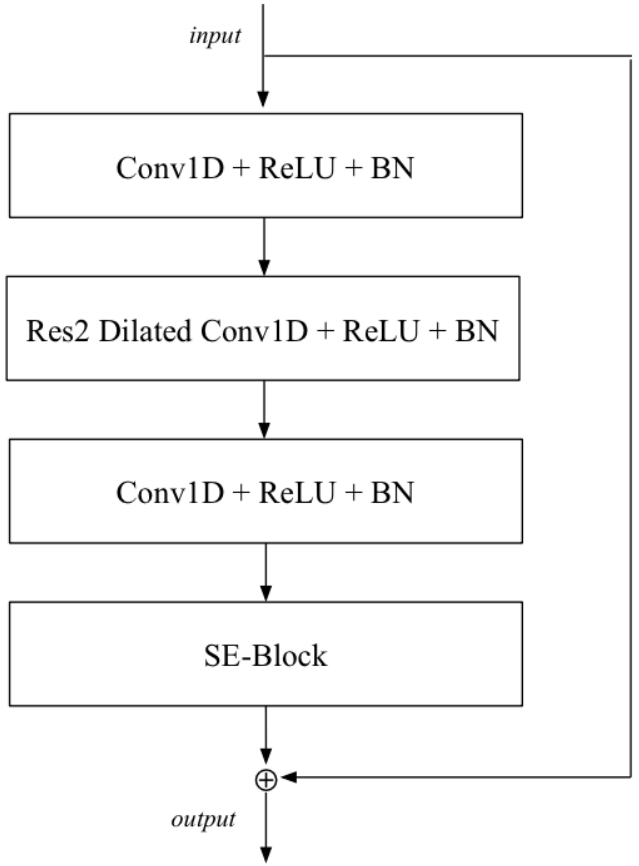
BEYOND TRANSCRIPTION: WHO IS SPEAKING?

ECAPA-TDNN: Introduction

- Speaker-verification used to use the x-vector architecture (a Time Delay Neural Network, TDNN) to produce fixed-length embeddings from variable-length utterances
- However, improvements are possible by borrowing ideas from deep-learning in computer vision and face recognition: e.g., channel-attention, residual/skip connections, multi-scale processing.
- The authors propose ECAPA-TDNN to enhance TDNN architectures via three main axes: **Channel Attention**, **Feature Propagation/Aggregation**, and a refined **Pooling** mechanism.
- In experiments, ECAPA-TDNN significantly outperforms strong baselines on the VoxCeleb1 / VoxSRC 2019 speaker-recognition tasks.

ECAPA-TDNN Architecture

- **1D SE-Res2Blocks**: Frame-level modules built from 1D dilated convs, residual connections, and Squeeze-and-Excitation blocks to model channel interdependencies and expand temporal context.
- **Multi-Layer Feature Aggregation (MFA) & Summation Residuals**: Instead of only using the last frame-layer output, ECAPA-TDNN concatenates or sums features from multiple levels to exploit hierarchical representations.
- **Channel- & Context-Dependent Attentive Statistics Pooling**: The pooling layer is augmented by channel-wise self-attention with global context (mean + std across time) to compute weighted mean and standard deviation per channel.



ECAPA-TDNN Detailed Mechanisms

- **Channel-Dependent Attention in Pooling**: For each channel c and time step t:

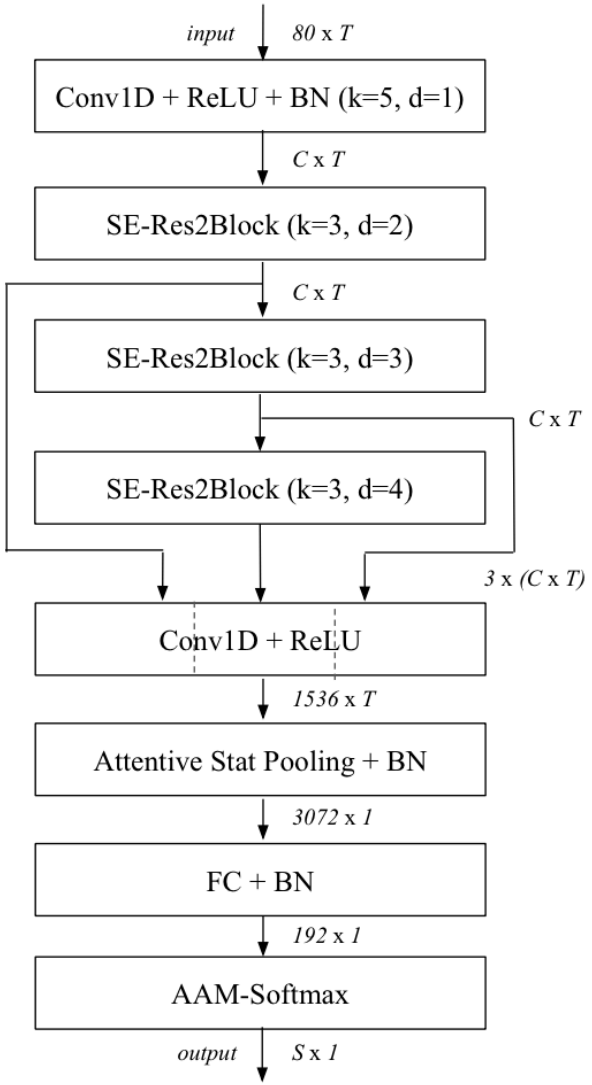
$$e_{t,c} = v_c^T f(W, h_t + b) + k_c$$

then softmax across t (for each c):

$$\alpha_{t,c} = \frac{\exp(e_{t,c})}{\sum_\tau \exp(e_{\tau,c})}$$

Weighted mean ($\tilde{\mu}_c$) and standard deviation ($\tilde{\sigma}_c$) are computed per channel.

- **SE Block (1-D)**:
  - Squeeze: $z_c = \frac{1}{T} \sum_t h_{t,c}$
  - Excitation: $s = \sigma(W_2 f(W_1 z + b_1) + b_2)$ then $\tilde{h}_c = s_c, h_c$
- **Res2Net Module**: Within each Res2Block the central conv is replaced by a multi-scale "Res2Net" style block (scale dimension $s = 8$) to capture different temporal granularities.



ECAPA-TDNN: Why it matters and what it was trained on (originally)

- **Data & Training**
  - Training on VoxCeleb2 dev set (5994 speakers) under the VoxSRC 2019 fixed-condition protocol.
  - Input features: 80-dim MFCCs (25 ms window, 10 ms shift), 2 s random crops, no VAD; augmented with MUSAN noise, RIR, tempo/codec, SpecAugment.
  - Optimisation: Cyclical learning rate (1e-8 → 1e-3), Adam, AAM-softmax margin 0.2, batch size 128.
  - For C = 512 channels: EER on VoxCeleb1 down to ~1.01 % (vs ~1.49 % baseline)
  - For C = 1024: EER ~0.87% and further improvements in MinDCF.
  - Ablation studies show: removing SE blocks or Res2Net modules or MFA degrades performance meaningfully.

ECAPA-TDNN: Summary & Take-aways

- The ECAPA-TDNN architecture effectively enhances TDNN-based speaker embedding extraction by emphasising channel attention (SE blocks + channel-wise attentive pooling), propagation (summed residuals) and aggregation (multi-layer feature fusion).
- Gains come with relatively modest parameter growth (e.g., C = 512 version has ~6.2 M parameters) while achieving state-of-the-art performance.
- **Practical take-aways for implementation**:
  - The attentive statistics pooling **should** consider both channel and temporal dimensions.
  - Multi-scale and multi-layer feature fusion (e.g., Res2Net + MFA) help capture richer speaker characteristics.
  - Data augmentation and training protocol remain critical for competitive performance in speaker verification.

Conclusion:

ECAPA-TDNN was designed for speaker recognition, but is also widely used for other task

- moderate number of parameters
- easy to adapated to other tasks
- embeddings can be useful out of the box

Other cool models and applications we didn't talk about:

Unsupervised Speech Recognition

AudioSpectrogramTransformer

Best-RQ

OWSM-CTC (open Whisper-style models)

Streaming / on-device decoding tricks

Speech foundation models (SeamlessM4T, USM)

Audio-Flamingo-3

X-Vectors / ECAPA-TDNN

Speaker Diarization

TTS

Speech Translation

Speech Emotion Recognition / Paralinguistics...

Sources

- Talk by Awni Hannun, Speech Recognition and Graph Transformer Networks
- Connectionist Temporal Classification : Labelling Unsegmented Sequence Data with Recurrent Neural Networks°, Graves, et al. 2006, ICML
- Hannun. 2017, Sequence Modelling with CTC, Distill, https://distill.pub/2017/ctc/
- Juang, B.H., Rabiner, L.R., 2004. Automatic Speech Recognition – A Brief History of the Technology Development.
- Rastorgueva, Elena, How does forced alignment work? [WWW Document]. URL https://research.nvidia.com/labs/conv-ai/blogs/2023/2023-08-forced-alignment/None (accessed 6.15.24).
- Alexei Baevski: https://icml.cc/virtual/2020/7203
- https://slideslive.com/38930744/selfsupervised-learning-of-speech-representations-with-wav2vec
- Graves, A., 2012. Sequence Transduction with Recurrent Neural Networks (RNN-T).
- Gulati, A. et al., 2020. Conformer: Convolution-augmented Transformer for Speech Recognition, Interspeech 2020.
- Hsu, W.-N. et al., 2021. HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units.
- Chen, S. et al., 2022. WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing.
- Radford, A. et al., 2022. Robust Speech Recognition via Large-Scale Weak Supervision (Whisper).
- Chu, Y. et al., 2023/2024. Qwen-Audio / Qwen2-Audio. · Tang, C. et al., 2024. SALMONN: Towards Generic Hearing Abilities for LLMs.